

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingenieria
Departamento de Ciencia de la Computacion

CC3086 — Programacion de Microprocesadores
Computacion Paralela y Distribuida
Semestre 2, 2026

Proyecto No. 1

Plinko 3D Paralelo

*Screensaver con simulacion fisica en C++ y paralelizacion de memoria
compartida con OpenMP*

Integrantes

Ian Rodrigo Cumes — 23236

Javier Valladares — 23045

Nery Molina — 23218

Guatemala, 30 de agosto de 2026

Indice

1. Introduccion	3
2. Antecedentes	3
3. Objetivos	4
3.1 Objetivo general	4
3.2 Objetivos especificos	4
4. Marco teorico	4
4.1 Memoria compartida y OpenMP	4
4.2 Metodo PCAM	5
4.3 Speedup, eficiencia y sus limites	5
5. Metodologia	6
5.1 Entorno de medicion	6
5.2 Protocolo de medicion	6
5.3 Por que se reporta el tiempo minimo y no el promedio	7
6. Diseno e implementacion	9
6.1 El problema y por que es paralelizable	9
6.2 Aplicacion del metodo PCAM	9
6.3 Doble buffer: por que no hay condiciones de carrera	9
6.4 Generacion pseudoaleatoria sin estado compartido	10
6.5 Las cuatro versiones	10
6.6 Mecanismos de proteccion de memoria compartida	11
6.7 Parametrizacion y programacion defensiva	12
6.8 Verificacion automatizada	12
7. Resultados	14
7.1 Panorama general	14
7.2 Eficiencia	15
7.3 Escalamiento con la cantidad de pelotas	16
7.4 Cuantas pelotas caben en 60 cuadros por segundo	17

7.5 Comparacion directa de las dos versiones de OpenMP	18
7.6 Dispersion de las mediciones	19
8. Analisis y discusion	21
8.1 Por que fracasa un hilo por pelota	21
8.2 El techo de seis hilos y los nucleos asimetricos	21
8.3 El tamano del problema decide si vale la pena paralelizar	22
8.4 Costo y beneficio de cada iteracion	22
8.5 Limitaciones del estudio	23
9. Conclusiones	24
10. Recomendaciones	24
11. Bibliografia	25
Anexo 1. Diagrama de flujo del programa	27
Anexo 2. Catalogo de funciones	30
Anexo 3. Bitacora de pruebas	40
A3.1 Resultados por version	40
A3.2 Mediciones individuales	42
A3.3 Captura de la salida del banco de pruebas	43
A3.4 Resultado de las pruebas automatizadas	44
Anexo 4. El programa en ejecucion	45

1. Introduccion

Este informe documenta el diseno, la implementacion y la evaluacion de **Plinko 3D Paralelo**, un screensaver escrito en C++17 que simula una piramide de Plinko tridimensional: N pelotas caen sobre el vertice de un cono de clavijas oscilantes, chocan entre si, se reparten hacia afuera rebotando de nivel en nivel, atraviesan zonas que alteran su fisica y terminan contadas en sectores angulares alrededor de la base. La camara orbita alrededor de la piramide, de modo que la estructura se percibe en tres dimensiones. El programa corre a pantalla completa, se dibuja con SDL2 y OpenGL, y despliega en pantalla los cuadros por segundo.

El objetivo del proyecto no es el juego en si, sino usarlo como banco de trabajo para un ejercicio de paralelizacion. Partimos de una version secuencial funcional y la fuimos transformando en versiones paralelas de memoria compartida, midiendo en cada iteracion el speedup y la eficiencia obtenidos. El resultado es una comparacion de cuatro estrategias sobre exactamente el mismo trabajo: una secuencial, una con un `std::thread` por pelota, y dos con OpenMP.

Lo que hace interesante al problema es que la parte cara de cada cuadro son las colisiones entre pelotas, cuyo costo crece con el cuadrado de N. Eso da suficiente trabajo por cuadro como para que la paralelizacion tenga algo que ganar, y al mismo tiempo obliga a resolver el problema central de la memoria compartida: como dejar que varios hilos lean el estado de todas las pelotas mientras cada uno escribe la suya, sin condiciones de carrera y sin pagar el costo de un candado por pelota.

El mejor resultado obtenido fue un speedup de **5.15x** con 8 hilos y N = 4 000 pelotas (eficiencia 0.64). El hallazgo mas util del proyecto, sin embargo, fue negativo: la primera version paralela, la que asignaba un hilo por pelota, resulto mas de cinco veces *mas lenta* que la secuencial (0.24x en el peor caso medido). Entender por que es lo que motivo el cambio a OpenMP.

2. Antecedentes

La programacion paralela de memoria compartida existe desde que los procesadores dejaron de ganar velocidad por reloj y empezaron a ganarla por cantidad de nucleos. Antes de OpenMP, aprovechar esos nucleos significaba escribir codigo explicito de hilos: crearlos, repartirles trabajo, sincronizarlos y destruirlos a mano, con una biblioteca distinta segun el sistema operativo.

OpenMP aparecio en 1997 como un estandar de directivas de compilador que permite expresar el paralelismo sin reescribir el programa. Su propuesta es incremental: se parte de un programa secuencial correcto y se le agregan directivas `#pragma omp` donde conviene, de modo que el mismo codigo sigue compilando y funcionando en un compilador que no soporte OpenMP (Chapman, Jost y van der Pas, 2007). Esa propiedad es exactamente la que este proyecto explota: las cuatro versiones conviven en el mismo binario y se pueden alternar con una tecla mientras el programa corre.

Para decidir *que* paralelizar, el proyecto sigue el metodo PCAM propuesto por Foster (1995): particionar el problema en tareas, analizar la comunicacion entre ellas, aglomerarlas para reducir el costo de esa

comunicacion y finalmente mapearlas a procesadores. Es un metodo pensado para memoria distribuida, pero sus cuatro etapas se aplican igual de bien a memoria compartida, donde la etapa de comunicacion se traduce en accesos a memoria y en sincronizacion.

Como antecedente directo, este proyecto continua la primera entrega del curso, en la que el equipo construyo un tablero con doce pelotas, gravedad y rebotes contra los bordes, y una version paralela con un `std::thread` persistente por pelota. Aquella entrega ya dejaba anotado que, con solo doce pelotas y una fisica sencilla, era posible que el modo paralelo resultara mas lento que el secuencial por el costo de sincronizacion. Este informe cuantifica esa sospecha y la resuelve.

3. Objetivos

3.1 Objetivo general

Disenar, implementar y evaluar un screensaver con simulacion fisica cuyo nucleo de computo se paralelice con OpenMP sobre memoria compartida, midiendo la mejora obtenida en terminos de speedup y eficiencia.

3.2 Objetivos especificos

- Implementar una version secuencial correcta y funcional que sirva como referencia de resultado y como denominador de todas las mediciones de speedup.
- Aplicar el metodo PCAM para identificar la descomposicion del problema y el patron de programacion paralela apropiado.
- Construir versiones paralelas sucesivas, cada una corrigiendo una limitacion medida en la anterior, y documentar la mejora que aporta cada iteracion.
- Emplear mecanismos de proteccion de memoria compartida y de sincronia adecuados a cada caso, evitando tanto las condiciones de carrera como la serializacion innecesaria.
- Parametrizar el programa mediante argumentos de linea de comandos con validacion defensiva, sin variables fijas en el codigo.
- Medir el tiempo de ejecucion con al menos diez repeticiones por configuracion y calcular el speedup y la eficiencia de cada version paralela.
- Determinar hasta que cantidad de elementos cada version sostiene 60 cuadros por segundo.

4. Marco teorico

4.1 Memoria compartida y OpenMP

En un sistema de memoria compartida todos los nucleos ven el mismo espacio de direcciones. Eso elimina el costo de enviar mensajes, pero introduce el problema opuesto: si dos hilos escriben la misma direccion sin coordinarse, el resultado depende del orden en que el planificador los ejecute. A eso se le llama

condicion de carrera, y su sintoma tipico es un programa que da resultados distintos en cada corrida.

OpenMP resuelve la coordinacion con un modelo de bifurcacion y union: al encontrar una region `#pragma omp parallel` el hilo maestro crea un equipo de hilos, todos ejecutan la region, y al cerrarla se sincronizan en una barrera implicita antes de que el maestro continue solo. Dentro de la region, `#pragma omp for` reparte las iteraciones de un ciclo entre los hilos del equipo, tambien con barrera implicita al final (OpenMP Architecture Review Board, 2021).

Las clausulas de reparto determinan como se dividen las iteraciones. Con `schedule(static)` cada hilo recibe de antemano un bloque contiguo del mismo tamano, lo que no cuesta nada en tiempo de ejecucion pero exige que todas las iteraciones cuesten lo mismo. Con `schedule(guided)` los hilos toman bloques que empiezan grandes y se van reduciendo, de modo que el reparto se ajusta solo cuando unas iteraciones resultan mas caras que otras, a cambio de un pequeno costo de planificacion.

4.2 Metodo PCAM

El metodo PCAM (Foster, 1995) descompone el diseno de un programa paralelo en cuatro etapas sucesivas:

- **Particion.** Dividir el problema en la mayor cantidad de tareas independientes posible, sin pensar todavia en cuantos procesadores hay.
- **Comunicacion.** Identificar que datos necesita cada tarea de las demas y con que frecuencia.
- **Aglomeracion.** Agrupar tareas pequenas en tareas mayores para reducir el costo de comunicacion y de creacion de tareas.
- **Mapeo.** Asignar las tareas aglomeradas a los procesadores disponibles, buscando equilibrar la carga.

La etapa de aglomeracion es la que explica el fracaso de nuestra primera version paralela: repartir una tarea por pelota es una particion correcta, pero saltarse la aglomeracion y mapear cada tarea a su propio hilo del sistema operativo hace que el costo de gestionar las tareas supere al trabajo que contienen.

4.3 Speedup, eficiencia y sus limites

El **speedup** compara el tiempo de la version secuencial contra el de la paralela sobre el mismo problema: $S(p) = T(1) / T(p)$, donde p es la cantidad de hilos. La **eficiencia** normaliza ese numero por la cantidad de recursos empleados: $E(p) = S(p) / p$. Una eficiencia de 1.0 significa que cada hilo agregado aporta exactamente el trabajo de un hilo; en la practica siempre es menor, porque una parte del programa no se paraleliza y porque la sincronizacion cuesta.

La **ley de Amdahl** (Amdahl, 1967) pone el techo: si una fraccion f del programa es inherentemente secuencial, el speedup no puede pasar de $1 / (f + (1-f)/p)$, y cuando p tiende a infinito el limite es $1/f$. Con $f = 5\%$ el techo es 20x, sin importar cuantos nucleos se agreguen.

La **ley de Gustafson** (Gustafson, 1988) matiza ese pesimismo observando que, en la practica, cuando se dispone de mas nucleos no se resuelve el mismo problema mas rapido sino un problema mas grande en el mismo tiempo. Esa observacion describe bien lo que ocurre en este proyecto: la parte secuencial de nuestro

paso de física es prácticamente constante, mientras que la parte paralela crece con N^2 , de modo que la eficiencia mejora conforme aumenta la carga. Es exactamente lo que muestran las mediciones: con $N = 250$ el mejor speedup apenas llega a 1.93x, mientras que con $N = 8\,000$, donde un solo paso secuencial cuesta 137 ms, alcanza 5.15x.

5. Metodología

El enunciado del proyecto define el alcance de manera compacta. Se pide, textualmente:

*“realizar un programa que dibuje en pantalla un ‘screensaver’ y que corra de forma paralela. Comenzaran diseñando y programando la version secuencial. Una vez su version secuencial este lista (funcional y corriendo), procederan a buscar acelerarla y mejorarla utilizando OpenMP”.*¹

El trabajo siguió ese orden. Primero se construyó la simulación secuencial completa y se verificó que produjera una escena estable; solo después se introdujeron las directivas de OpenMP. Cada versión paralela se aceptó únicamente cuando dos condiciones se cumplieran: que produjera exactamente el mismo estado que la secuencial, y que las pruebas automatizadas de determinismo pasaran con distintas cantidades de hilos.

5.1 Entorno de medicion

Todas las mediciones reportadas se tomaron en la misma máquina y en la misma sesión, con el binario compilado en modo Release:

Tabla 1. Entorno de las mediciones.

Componente	Detalle
Procesador	Apple M1 Pro, 8 núcleos (6 de rendimiento y 2 de eficiencia)
Memoria	16 GB unificados
Sistema operativo	macOS 26.5.1 (25F80)
Compilador	Apple clang 17.0.0, C++17, -O3 (CMAKE_BUILD_TYPE=Release)
OpenMP	libomp 5.1 (Homebrew), enlazada mediante -Xpreprocessor -fopenmp
Graficos	SDL2 y OpenGL 2.1 en perfil de compatibilidad

El detalle del procesador no es un dato de relleno: el M1 Pro es un procesador *heterogeneo*, con seis núcleos rápidos y dos lentos. Esa asimetría explica buena parte de los resultados de la sección 7 y es la razón por la que la eficiencia cae al pasar de seis a ocho hilos.

5.2 Protocolo de medicion

El programa incluye un modo de banco de pruebas (- -benchmark) que ejecuta exactamente la misma física que el screensaver pero sin abrir ventana ni dibujar nada. Eso es deliberado: si se midiera el cuadro completo, el tiempo estaría dominado por el renderizado y por la sincronía vertical, y el efecto de la paralelización quedaría oculto. El protocolo de cada medición es el siguiente:

- Se usa un paso de tiempo fijo de 1/60 de segundo, en lugar del reloj real, para que dos corridas simulen exactamente el mismo trabajo.
- Antes de cronometrar se ejecutan tres pasos de calentamiento, que llenan las caches y obligan a OpenMP a crear su equipo de hilos, de modo que ese costo no contamine la primera toma.
- Se cronometran 12 repeticiones por configuracion, dos mas que el minimo exigido, y se reportan promedio, minimo, maximo y desviacion estandar.
- La cantidad de pasos por repeticion se ajusta de forma inversamente proporcional a N^2 , para que las cargas grandes no dominen el tiempo total del banco.
- El speedup se calcula siempre contra la version secuencial medida con el mismo N en la misma corrida, no contra un valor de referencia guardado.

5.3 Por que se reporta el tiempo minimo y no el promedio

Las cifras de este informe usan el **menor** de los doce tiempos de cada configuracion, no el promedio. La decision merece explicarse porque no es la mas intuitiva.

El equipo no dispone de una maquina dedicada: las mediciones se tomaron en un equipo de trabajo con macOS, que mantiene procesos de fondo fuera del control del programa. Esa interferencia tiene una propiedad importante: solo puede *anadir* tiempo a una medicion, nunca quitarlo. El minimo es, por lo tanto, la mejor estimacion disponible del costo real del codigo, mientras que el promedio mezcla el costo del programa con el de lo que haya estado corriendo al lado.

No es una eleccion de conveniencia, y hay una comprobacion que lo demuestra. La version de OpenMP ejecutada con **un solo hilo** hace exactamente el mismo trabajo que la secuencial, de modo que su speedup tiene que dar 1.00 por construccion; cualquier desviacion es ruido de medicion y nada mas. Con el estimador del minimo, esa fila da 1.00 o 1.01 en las seis cargas y en las dos versiones. Con el promedio se dispersaba entre 0.90 y 1.06, e incluso llegaba a producir eficiencias superiores a 1.0, que serian fisicamente imposibles en este programa.

En una primera corrida el efecto fue extremo: las primeras cuatro repeticiones de la version secuencial con $N = 2\,000$ dieron entre 15 y 24 ms mientras las ocho restantes se estabilizaron en 10.5 ms. Ese unico bloque contaminado inflaba el promedio de la referencia y, con el, todos los speedup calculados contra ella. El minimo no se vio afectado.

Nada se oculta con esto: el Anexo 3 reproduce las doce repeticiones individuales de cada configuracion, junto con el promedio, el maximo y la desviacion estandar, de modo que cualquiera puede recalculas las cifras con el estimador que prefiera. La seccion 7.6 discute justamente la dispersion observada.

Se barrieron seis valores de N (250, 500, 1 000, 2 000, 4 000 y 8 000) y cinco cantidades de hilos (1, 2, 4, 6 y 8), lo que da 72 configuraciones y 828 mediciones individuales. Todas estan en el Anexo 3 y en los archivos `docs/resultados/` del repositorio.

¹ Universidad del Valle de Guatemala, Departamento de Ciencia de la Computacion. *Proyecto No. 1*, Computacion Paralela y Distribuida, Semestre 2, 2026, seccion “Contenido”.

6. Diseño e implementación

6.1 El problema y por qué es paralelizable

Cada cuadro de la simulación consiste en avanzar N pelotas un intervalo de tiempo. Para una pelota concreta ese avance requiere: aplicar gravedad y el efecto de las K zonas modificadoras que la contienen, resolver las colisiones contra las otras $N-1$ pelotas, integrar velocidad y posición, resolver las colisiones contra las M clavijas de la pirámide, rebotar contra la pared cilíndrica y el techo, y finalmente comprobar si llegó al piso para anotarla en un sector angular y hacerla reaparecer sobre el vértice.

El costo total por paso es entonces $O(N^2 + N \cdot M + N \cdot K)$. El término dominante para N grande es el cuadrático de las colisiones entre pelotas, y es precisamente el que interesa paralelizar: es trabajo aritmético puro, sin entrada ni salida, y crece rápido con el parámetro que el usuario controla.

6.2 Aplicación del método PCAM

Tabla 2. Descomposición del problema según el método PCAM.

Etapa	Decisión tomada
Partición	Una tarea por pelota. Es la descomposición de dominio natural: el arreglo de pelotas es el único que crece con el parámetro N que el usuario controla, y cada pelota se actualiza con la misma secuencia de operaciones.
Comunicación	Cada tarea necesita leer el estado del cuadro anterior de todas las demás pelotas, de las clavijas y de los modificadores, pero solo escribe su propia posición. Es un patrón de recolección (gather): muchas lecturas compartidas, una sola escritura privada.
Aglomeración	Las tareas se agrupan en bloques de índices contiguos. Como el arreglo de pelotas está contiguo en memoria, un bloque de índices es también un bloque de líneas de caché, lo que aprovecha la localidad espacial y amortiza el costo de planificación entre muchas pelotas.
Mapeo	Los bloques se asignan a los hilos del equipo de OpenMP. Se probaron dos políticas: reparto estático, que no cuesta nada pero supone carga uniforme, y reparto guiado, que se adapta cuando unas pelotas colisionan más que otras o cuando los núcleos no son igual de rápidos.

6.3 Doble buffer: por qué no hay condiciones de carrera

La decisión de diseño más importante del proyecto es la que elimina las carreras por construcción en lugar de protegerlas con candados. La simulación mantiene *dos* arreglos de pelotas: `current_`, que durante todo el paso es de solo lectura, y `next_`, en el que cada índice tiene un único escritor. Al terminar el paso, los dos se intercambian.

De ahí se siguen tres propiedades que valen la pena enumerar. Primero, ningún hilo escribe una dirección que otro pueda leer durante el paso, de modo que no hace falta ningún candado sobre el estado de las pelotas. Segundo, como todas las tareas leen el mismo estado congelado, el resultado no depende del orden en que los hilos las ejecuten: la simulación es *determinista* y produce exactamente los mismos bits con 1 hilo que con 16. Tercero, esa determinación se puede convertir en una prueba automatizada, y es lo que hacen las suites *equivalencia* y *determinismo* descritas en la sección 6.8.

La alternativa habitual —actualizar las pelotas en el mismo arreglo y proteger cada par con un candado— habría costado un mutex por colisión, es decir del orden de N^2 operaciones atómicas por paso, y además

habria hecho el resultado dependiente del orden de ejecucion. El costo del doble buffer es la memoria: dos arreglos en lugar de uno. Con 8 000 pelotas de 56 bytes, eso son 896 KB en total, un precio despreciable.

6.4 Generacion pseudoaleatoria sin estado compartido

Un detalle que suele romper el determinismo de un programa paralelo es el generador de numeros aleatorios. Usar `std::rand` o un unico `std::mt19937` global obligaria a serializar con un mutex y, peor aun, haria que el resultado dependiera del orden en que los hilos consumen numeros.

La solucion adoptada es que cada pelota lleve su propia semilla de 32 bits dentro de su estructura, y que la avance con un generador SplitMix32 sin estado global. La semilla inicial de la pelota i se deriva de la semilla global y de i mediante una funcion de mezcla, de modo que la inicializacion tampoco necesita recorrer el generador secuencialmente. El resultado es que la desviacion pseudoaleatoria que recibe una pelota al golpear una clavija depende unicamente de su propia historia, nunca del calendario de los hilos.

6.5 Las cuatro versiones

Las cuatro estrategias comparten el mismo nucleo de fisica, la funcion `advanceBall()`, y se diferencian unicamente en como recorren el arreglo de pelotas y en como acumulan los contadores compartidos. Eso garantiza que la comparacion sea justa: se esta midiendo la estrategia de paralelizacion, no dos implementaciones distintas de la fisica.

Version 0 — Secuencial

Un ciclo simple sobre las N pelotas, en un solo hilo, que acumula los sectores en el mismo recorrido. Es la referencia de correccion y el denominador de todo speedup reportado en este informe.

Version 1 — Un `std::thread` por pelota

Esta fue la primera version paralela del equipo, heredada de la entrega anterior. Crea N hilos persistentes, uno por pelota, y los coordina con una barrera de dos fases construida sobre un mutex y dos variables de condicion: el hilo coordinador publica los datos de la ronda, incrementa un numero de generacion y despierta a todos; cada worker calcula su pelota fuera de la seccion critica y decrementa un contador de pendientes; el ultimo en terminar despierta al coordinador.

La implementacion es correcta —pasa la prueba de equivalencia— pero es un desastre de rendimiento, y por eso se conserva en el codigo final: es el contraejemplo que justifica todo lo demas. Con $N = 1\,000$ el programa crea mil hilos del sistema operativo que, en cada uno de los dos sub-pasos de cada cuadro, deben despertarse, competir por un unico mutex, hacer unos pocos microsegundos de trabajo y volver a dormirse. Ese patron se conoce como “estampida de hilos” y su costo crece linealmente con N mientras el trabajo por hilo permanece constante. Por encima de 1 024 pelotas el sistema operativo sencillamente rechaza la creacion; el programa detecta la excepcion, informa y continua con otra estrategia en lugar de abortar.

Version 2 — OpenMP con reparto estatico

La traduccion directa del ciclo secuencial. Una sola directiva convierte el recorrido en paralelo:

```
#pragma omp parallel for schedule(static) num_threads(threads) \
    shared(readBalls, writeBalls, pegData, modifierData, binData, params) \
    firstprivate(elements, pegCount, modifierCount, subTime, subDelta) \
    reduction(+ : recycled)
```

Los punteros a los buffers se copian a variables locales antes de la region para no depender del puntero `this` dentro de las clausulas, lo que hace el codigo mas portable entre compiladores y evita una indireccion en el ciclo caliente. El total de pelotas recicladas se acumula con una reduccion, y los contadores por sector —que si son un recurso realmente compartido— se protegen con `#pragma omp atomic`.

Version 3 — OpenMP ajustado

La tercera version corrige tres cosas que la medicion de la segunda dejo en evidencia:

- **Una sola region paralela por cuadro.** La version estatica abria y cerraba una region por cada sub-paso de integracion. La ajustada abre `#pragma omp parallel` una vez y ejecuta dentro de ella todos los sub-pasos, usando la barrera implicita del `#pragma omp for` como sincronia entre uno y otro. La alternancia de buffers se resuelve por la paridad del sub-paso, porque dentro de la region no es posible intercambiar los vectores.
- **Reparto guiado.** Con `schedule(guided)` los bloques empiezan grandes y se reducen, lo que nivela la carga cuando unas pelotas colisionan mas que otras y, sobre todo, cuando dos de los ocho nucleos son mas lentos que los otros seis.
- **Contadores privatizados.** En lugar de una operacion atomica por pelota reciclada, cada hilo acumula en un arreglo privado y al final del cuadro fusiona su copia dentro de un `#pragma omp critical`. La seccion critica se ejecuta una vez por hilo y por cuadro, no una vez por pelota.

Esta version incluye ademas una metrica de diagnostico: cada hilo registra su tiempo ocupado en una entrada alineada a 64 bytes —para que dos hilos nunca escriban en la misma linea de cache y no aparezca *false sharing*— y, tras un `#pragma omp barrier` explicito, un solo hilo calcula el cociente entre el tiempo maximo y el promedio. Ese numero es el desbalance de carga, y es lo que permitio identificar la asimetria de los nucleos como la causa de la caida de eficiencia con ocho hilos.

6.6 Mecanismos de proteccion de memoria compartida

El proyecto usa cuatro mecanismos distintos, cada uno donde corresponde. Vale la pena listarlos juntos porque la eleccion, y no solo la presencia, es lo que distingue una paralelizacion buena de una que apenas funciona:

Tabla 3. Mecanismos de sincronia empleados y su justificacion.

Mecanismo	Donde	Por que ahi
Barrera de dos fases con mutex y dos condition_variable	BallThreadSystem::runRound	Es el unico modo de coordinar hilos crudos de C++. El coordinador no avanza hasta que el contador de pendientes llega a cero, lo que impide que OpenGL lea una posicion que otro hilo todavía escribe.

Mecanismo	Donde	Por que ahi
Barrera implicita de <code>#pragma omp for</code>	Entre sub-pasos de integracion	El sub-paso siguiente lee lo que el actual acaba de escribir, así que la barrera es obligatoria. Aprovechar la implicita evita agregar una explicita redundante.
<code>#pragma omp atomic</code>	Contadores de sector en la version estatica	Varias pelotas pueden caer en el mismo sector en el mismo paso. El incremento atomico es mas barato que una seccion critica cuando la contencion es baja.
<code>#pragma omp critical</code> + contadores privados	Fusion de sectores en la version ajustada	Privatizar y fusionar una vez por hilo cambia el costo de O(reciclajes) operaciones atomicas a O(hilos) secciones criticas por cuadro.
<code>reduction(+ : ...)</code>	Total de pelotas recicladas	La reduccion la implementa el compilador con acumuladores privados y una combinacion en arbol; es la forma idiomática y la mas eficiente.
<code>#pragma omp barrier</code> + <code>#pragma omp single</code>	Metrica de desbalance de carga	Todos los hilos deben haber publicado su tiempo antes de que uno solo lo reduzca. Aqui la barrera explicita si es necesaria.

6.7 Parametrizacion y programacion defensiva

El proyecto exige evitar variables fijas en el código, y el programa no tiene ninguna que afecte la carga de trabajo o el lienzo: todo se lee de la línea de comandos. Las opciones cubren la escena (N, niveles y anchura de la piramide, cantidad de modificadores y de sectores, radio y altura del cilindro, radio de la pelota, gravedad, restitution, sub-pasos, semilla), la ventana (ancho, alto, pantalla completa, sincronia vertical), la camara (velocidad de giro e inclinacion), la ejecucion (modo e hilos) y la medicion (banco de pruebas y capturas).

Cada valor se valida antes de usarse. Las conversiones numericas exigen que se consuma toda la cadena, de modo que una entrada como `12abc` se rechaza en lugar de aceptarse parcialmente; se verifica que las opciones que llevan valor efectivamente lo reciban antes de leer fuera de `argv`; y todos los rangos se comprueban, incluido el minimo de 640x480 pixeles que pide el enunciado y el minimo de diez repeticiones del banco de pruebas. Ante cualquier error el programa explica cual fue el problema, imprime la ayuda y termina con código distinto de cero.

Si el usuario no indica N, el programa lo solicita por consola. Esa solicitud solo ocurre cuando la entrada estandar es interactiva —de lo contrario, dentro de un script o de CTest, el programa se quedaria esperando para siempre—, admite Enter para conservar el valor por omision y reintenta hasta tres veces antes de rendirse.

6.8 Verificacion automatizada

El repositorio incluye cuatro suites de pruebas registradas en CTest. Las dos primeras son las que sostienen todo el argumento de correccion del informe:

- **equivalencia:** ejecuta 180 pasos con 220 pelotas en los cuatro modos y exige que el estado final sea identico *bit a bit*, sin tolerancia. No se usa una comparacion aproximada a proposito: como el diseno garantiza que las operaciones de punto flotante se ejecutan en el mismo orden en todos los modos, cualquier diferencia indicaria una condicion de carrera real.

- **determinismo:** repite 240 pasos con 1, 2, 3, 5, 8 y 16 hilos y verifica que tanto el estado de las pelotas como los conteos por sector coincidan.
- **argumentos:** 21 casos de linea de comandos, entre validos e invalidos, que comprueban que la programacion defensiva acepta lo correcto y rechaza lo incorrecto con un mensaje explicativo.
- **conteos:** verifica que la suma de los sectores coincida exactamente con el total de pelotas recicladas. Detectaria un incremento perdido por una carrera en los contadores compartidos, que es justo lo que protegen `atomic` y `critical`.

Las cuatro suites pasan en la version entregada.

7. Resultados

Esta seccion presenta las cifras agregadas. Las 828 mediciones individuales que las sostienen estan en el Anexo 3.

7.1 Panorama general

Tabla 4. Resumen comparativo de las cuatro versiones. El speedup se calcula contra la version secuencial medida con el mismo N.

N	Secuencial ms/paso	std::thread speedup	OMP estatico 8h speedup	OMP ajustado 8h speedup	Mejor speedup observado	Hilos del mejor
250	0.411	0.26x	1.56x	1.31x	2.25x	4
500	1.082	0.25x	2.54x	2.35x	3.02x	4
1 000	3.117	0.24x	3.47x	3.98x	4.31x	6
2 000	10.095	no viable	3.76x	5.08x	5.08x	8
4 000	36.450	no viable	4.16x	5.15x	5.15x	8
8 000	133.428	no viable	4.48x	5.07x	5.07x	8

La tabla concentra los tres hallazgos del proyecto. Primero, la version de un hilo por pelota es consistentemente mas de cinco veces mas lenta que la secuencial: con $N = 1\,000$ su speedup es de apenas 0.24x, es decir que tarda 4.2 veces mas. Segundo, ambas versiones de OpenMP aceleran el programa en todos los tamanos probados, con un maximo de 5.15x. Tercero, cual de las dos versiones de OpenMP conviene depende del punto de operacion: hasta cuatro hilos son equivalentes, y la ventaja clara de la ajustada aparece con ocho hilos y carga alta (5.07x contra 4.48x con $N = 8\,000$). La seccion 8.2 desglosa el cuadro completo.

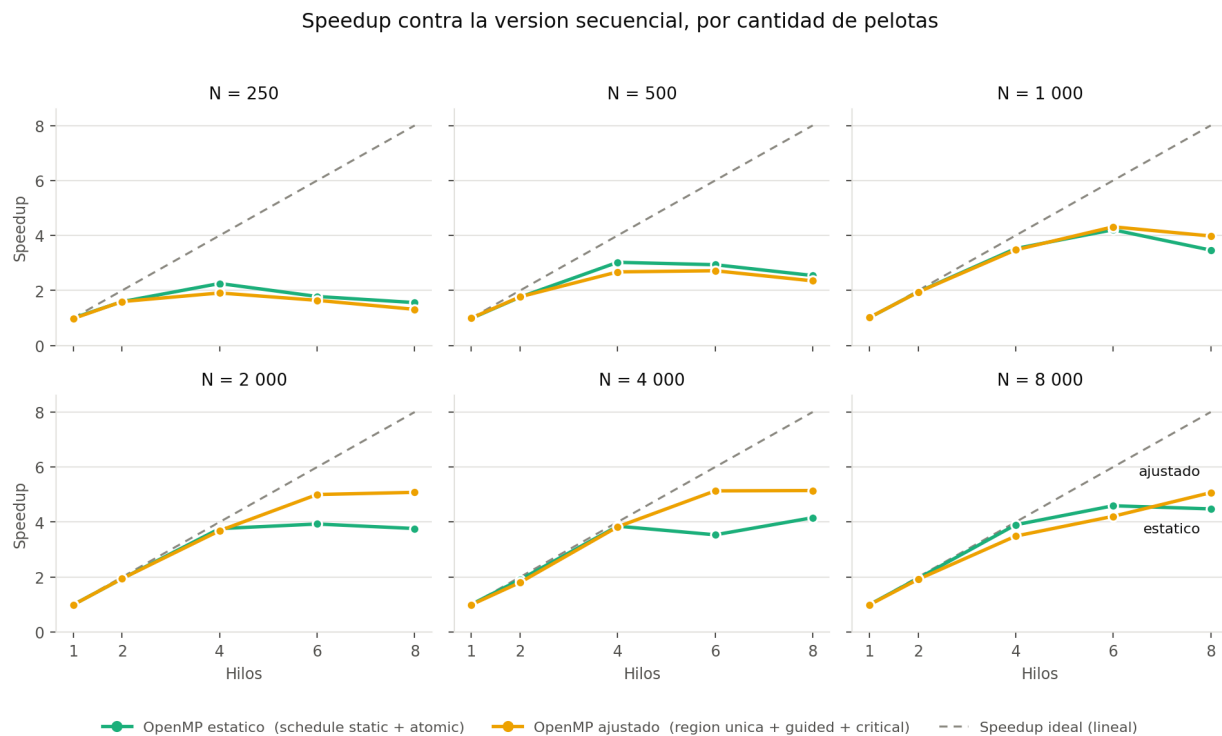


Figura 1. Speedup de las dos versiones de OpenMP frente al numero de hilos, con un panel por cantidad de pelotas. La linea discontinua marca el speedup ideal. La distancia entre las curvas y esa linea crece al reducir N, que es el comportamiento que anticipa la ley de Gustafson.

7.2 Eficiencia

La eficiencia es la cifra que mejor describe el aprovechamiento real del hardware. Los resultados dibujan un patron muy nitido: con dos y cuatro hilos, y siempre que N sea suficientemente grande, la eficiencia se mantiene por encima de 0.90; al pasar a seis y sobre todo a ocho hilos, cae de forma marcada.

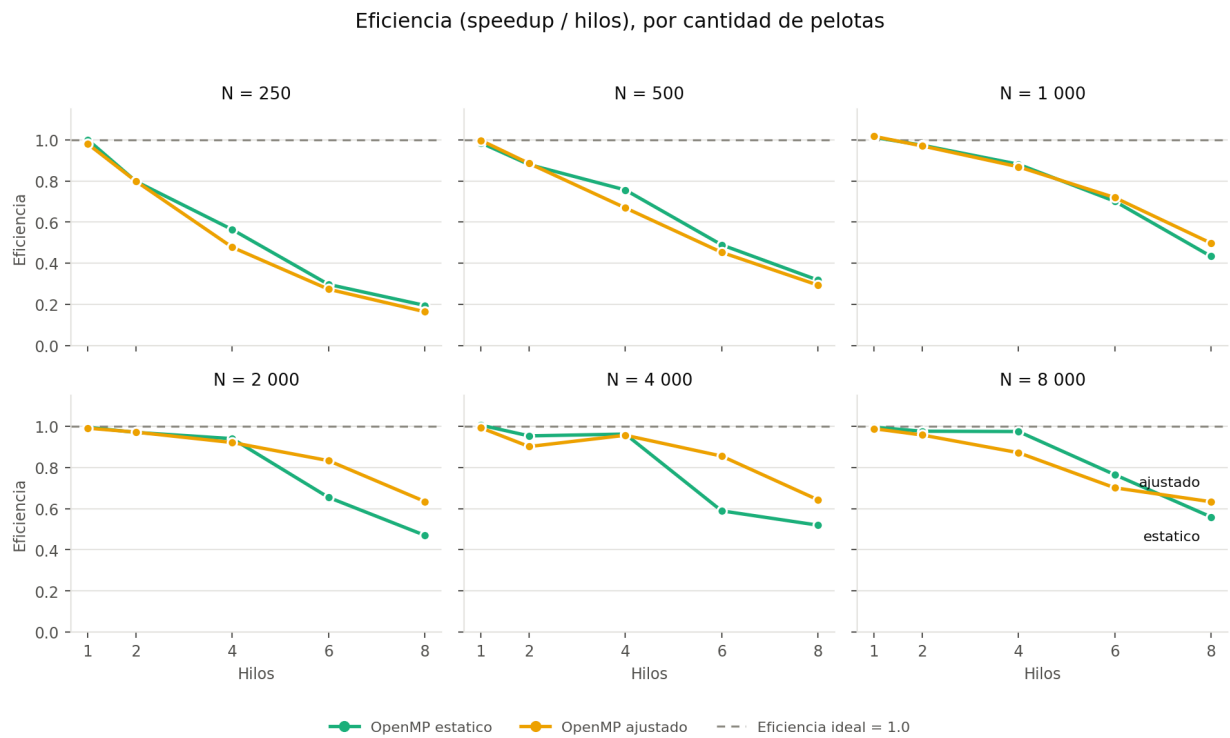


Figura 2. Eficiencia (speedup dividido entre el numero de hilos). El descenso a partir de seis hilos es sistematico y se explica en la seccion 8.

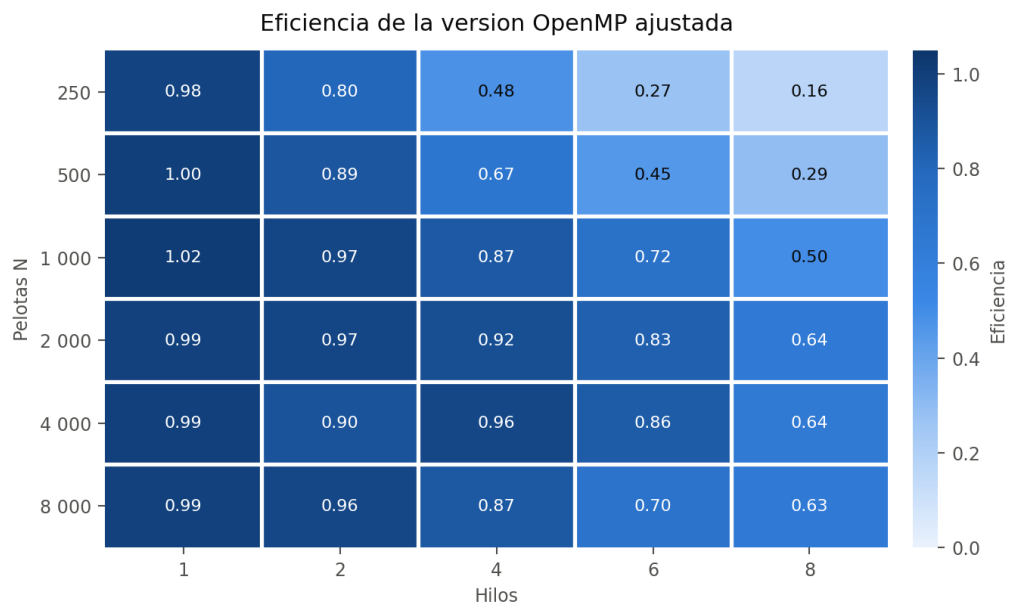


Figura 3. Eficiencia de la version OpenMP ajustada para cada combinacion de carga y de hilos. La region de eficiencia alta se ensancha hacia la derecha conforme crece N.

7.3 Escalamiento con la cantidad de pelotas

En escala logarítmica doble, un costo $O(N^2)$ aparece como una recta de pendiente 2. La pendiente medida entre $N = 250$ y $N = 8\,000$ es de 1.67, y entre $N = 1\,000$ y $N = 8\,000$ sube a 1.81. La diferencia es esperable: para N pequeño el costo lineal de las clavijas y los modificadores todavía pesa, y una parte del arreglo cabe en cache; conforme N crece, el término cuadrático de las colisiones entre pelotas domina cada vez más y la pendiente se acerca a 2.

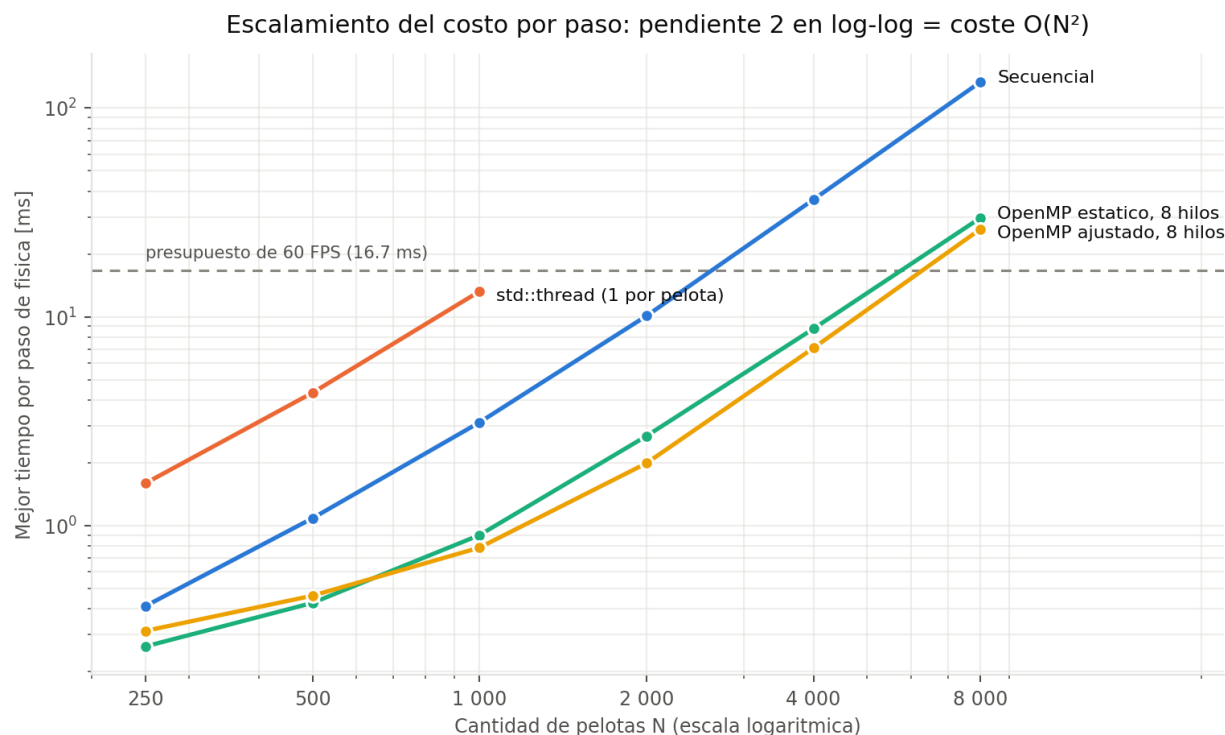


Figura 4. Tiempo por paso de física frente a N , en escala logarítmica doble. La línea discontinua es el presupuesto de 16.7 ms que corresponde a 60 cuadros por segundo.

7.4 Cuántas pelotas caben en 60 cuadros por segundo

El enunciado pide medir cuántos elementos se pueden generar sin perder cuadros por segundo. Tomando como presupuesto los 16.7 ms de un cuadro a 60 FPS y midiendo solo la física, los límites observados son los siguientes:

Tabla 5. Último valor de N medido que se mantiene dentro del presupuesto de 60 FPS, y primer valor que lo excede.

Version	Último N dentro del presupuesto	ms/paso ahí	Primer N que lo excede	ms/paso ahí
Secuencial	2 000	10.10	4 000	36.45
std::thread (1 por pelota)	1 000	13.23	no se alcanzó	—
OpenMP estático, 8 hilos	4 000	8.77	8 000	29.79
OpenMP ajustado, 8 hilos	4 000	7.08	8 000	26.32

En terminos practicos: la version secuencial se queda sin presupuesto entre 2 000 y 4 000 pelotas, mientras que la version ajustada con ocho hilos todavia cumple con 4 000. La paralelizacion, por lo tanto, duplica con holgura la cantidad de elementos que el screensaver puede mostrar sin perder fluidez.

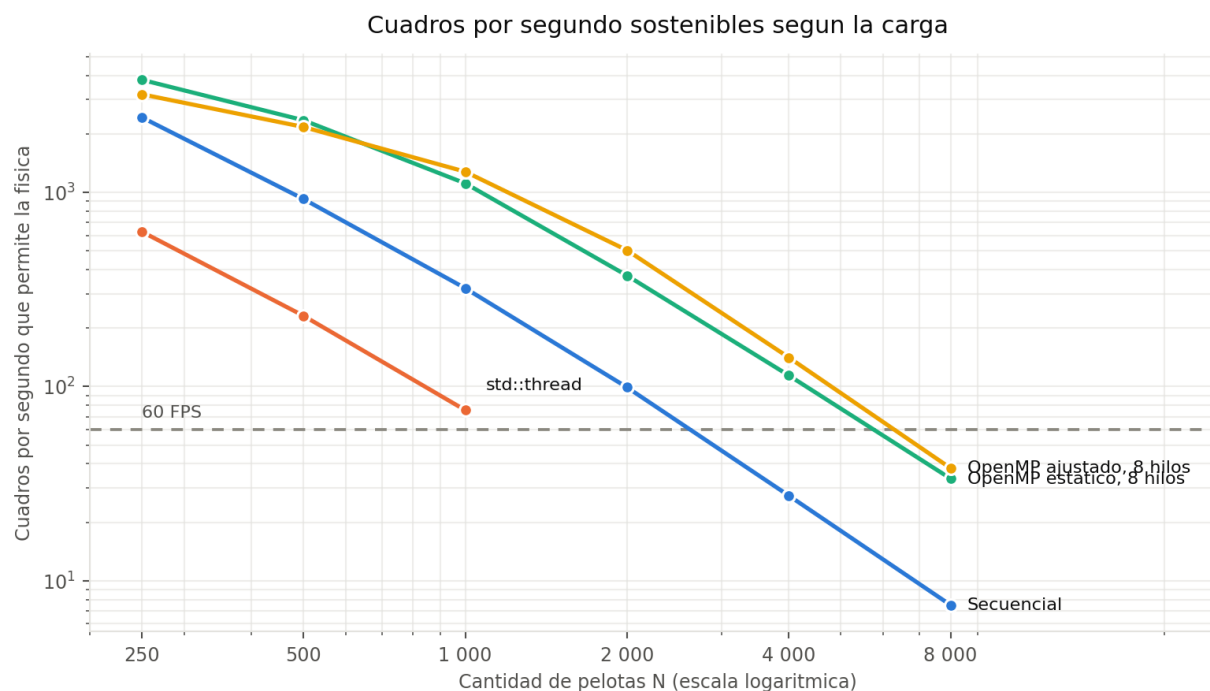


Figura 5. Cuadros por segundo que permite la fisica de cada version. Solo se contabiliza el costo de la simulacion, no el del dibujado.

7.5 Comparacion directa de las dos versiones de OpenMP

Con carga alta la version ajustada supera a la estatica precisamente donde la estatica se degrada: con ocho hilos y $N = 8\,000$ pasa de 4.48x a 5.07x, una mejora relativa del 13 %. Con cuatro hilos, en cambio, las dos son practicamente indistinguibles (3.90x contra 3.49x), lo que confirma que la ganancia del reparto guiado proviene de nivelar hilos desiguales, no de reducir la sobrecarga en general. Con cargas pequenas la relacion se invierte: el reparto guiado cobra un costo de planificacion que ahi no alcanza a amortizarse.

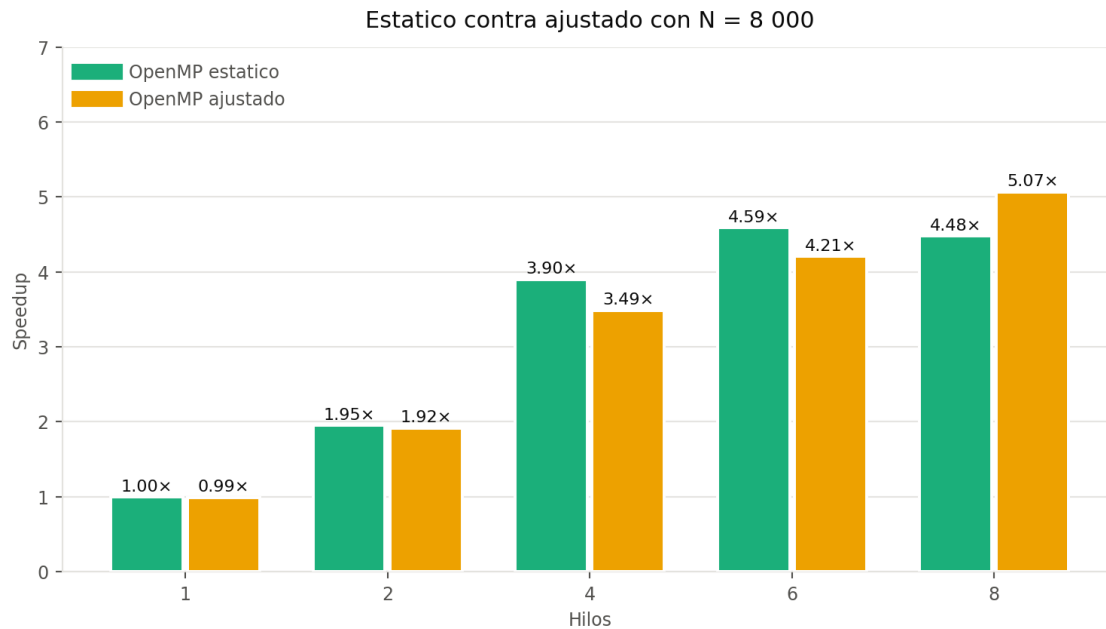


Figura 6. Speedup de ambas versiones de OpenMP con N = 8 000. Con cuatro hilos empatan; la diferencia aparece al usar los nucleos de eficiencia.

7.6 Dispersion de las mediciones

Las doce repeticiones de cada configuracion permiten juzgar cuanta confianza merece cada cifra. La forma de la distribucion es siempre la misma: un grupo compacto de valores bajos y unos pocos valores altos aislados. Esa asimetria es la firma tipica de la interferencia externa y es exactamente lo que justifica el criterio del minimo expuesto en la seccion 5.3: los valores altos no describen al programa, describen a lo que corria junto a el.

La dispersion crece de forma sistematica con la cantidad de hilos. Con uno, dos y cuatro hilos las repeticiones se agrupan estrechamente; con seis y ocho aparecen valores extremos. Tiene sentido: cuantos mas hilos pide el programa, mas probable es que alguno de ellos comparta un nucleo con un proceso del sistema, y el paso completo se retrasa hasta que ese hilo rezagado llega a la barrera.

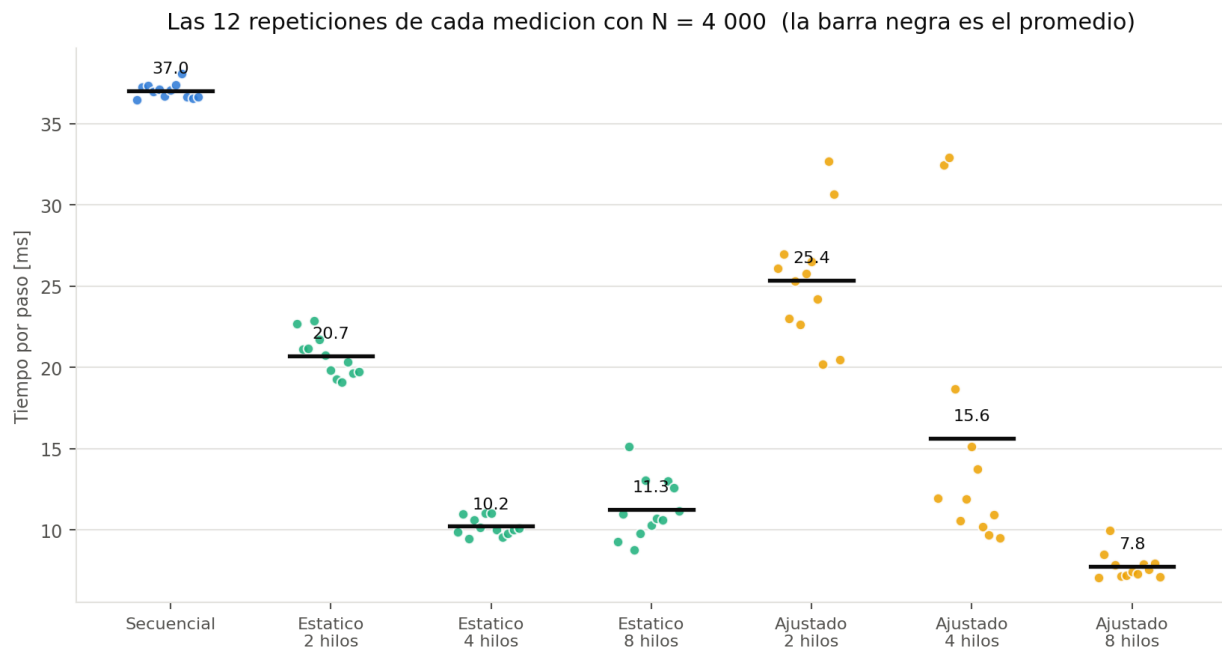


Figura 7. Las doce repeticiones de cada medicion con N = 4 000. Cada punto es una repeticion y la barra negra marca el promedio. La dispersion crece visiblemente al usar mas de cuatro hilos.

8. Analisis y discusion

8.1 Por que fracasa un hilo por pelota

El resultado mas contundente del proyecto es que la estrategia mas intuitiva —una tarea, un hilo— es la peor de todas. Conviene desarmar el numero. Con $N = 250$ el paso secuencial cuesta 0.411 ms, es decir unos 1.6 microsegundos por pelota. La version de hilos tarda 1.592 ms para el mismo trabajo. La diferencia no es computo: es el costo de despertar 250 hilos, hacerlos competir por un unico mutex, y volverlos a dormir, dos veces por cuadro.

Ese costo escala con N mientras el trabajo por hilo permanece constante, de modo que el speedup se estanca por debajo de 0.20x sin importar el tamano: 0.26x con $N = 250$ y 0.24x con $N = 1\,000$. Dicho de otra forma, el programa dedica alrededor del 76 % de su tiempo a coordinar hilos y el resto a simular.

En terminos de PCAM, el error esta en la etapa de aglomeracion: la particion en una tarea por pelota es correcta, pero mapear cada tarea a un hilo del sistema operativo salta la aglomeracion por completo. Los hilos son un recurso caro y limitado; las tareas, no. OpenMP hace exactamente lo que faltaba: agrupa muchas tareas en un bloque y le da ese bloque a un hilo de un equipo reutilizado.

Hay un limite adicional, mas duro: por encima de 1 024 pelotas el sistema operativo rechaza la creacion de hilos. El programa captura la excepcion, informa en pantalla y cambia de modo en lugar de abortar, pero la conclusion es que la estrategia sencillamente no escala al rango de N que el proyecto necesita.

8.2 El techo de seis hilos y los nucleos asimetricos

La observacion mas interesante de las mediciones es que la eficiencia no decae de forma suave, sino que se quiebra al pasar de cuatro a seis hilos y empeora con ocho. La explicacion esta en el hardware: el Apple M1 Pro no tiene ocho nucleos iguales, sino seis de rendimiento y dos de eficiencia, y estos ultimos son sustancialmente mas lentos.

Con `schedule(static)` las iteraciones se reparten en bloques identicos antes de empezar. Si dos de los ocho bloques caen en nucleos lentos, los otros seis hilos terminan y esperan en la barrera a que los rezagados acaben. El tiempo del paso queda determinado por el hilo mas lento, y la eficiencia se desploma: es un caso de libro de desbalance de carga, solo que provocado por el hardware y no por los datos.

El reparto guiado ataca justamente eso. Con $N = 8\,000$ y ocho hilos la version estatica se queda en 4.48x mientras la ajustada llega a 5.07x, porque los nucleos rapidos siguen tomando bloques mientras los lentos terminan el suyo. Aun asi la eficiencia no vuelve a los niveles de cuatro hilos (0.63 contra 0.87): agregar dos nucleos que rinden una fraccion de los otros seis nunca podra dar eficiencia 1.

Ahora bien, el reparto guiado no es gratis, y conviene ser preciso sobre cuando conviene, porque el resultado no es uniforme. Comparando las dos versiones celda a celda:

- **Con dos hilos** son indistinguibles: las diferencias no pasan de 0.10x, dentro del ruido de medicion.

- **Con cuatro hilos** la estatica es igual o ligeramente mejor en las seis cargas. Tiene sentido: cuatro hilos caben de sobra en los seis nucleos rapidos, no hay desbalance que corregir, y lo unico que aporta el reparto guiado es su costo de planificacion.
- **Con ocho hilos**, que es cuando los dos nucleos lentos entran al equipo, la ajustada gana en las cuatro cargas de N mayor o igual a 1 000, con una ventaja de hasta el 35 %.
- **Con seis hilos** el resultado es mixto: la ajustada gana en el rango medio (N entre 1 000 y 4 000) y la estatica en los extremos.
- **Con N pequeno** (250 y 500) la estatica gana con cuatro, seis y ocho hilos: cada bloque de trabajo es tan corto que pedir tarea de nuevo cuesta mas que lo que se gana nivelando.

La conclusion practica no es que una version sea mejor que la otra, sino que `schedule(guided)` es un seguro contra la heterogeneidad: se paga siempre y solo rinde cuando esa heterogeneidad existe. En un procesador homogeneo, o con cargas pequenas, el reparto estatico es la eleccion correcta.

Esta lectura se apoya en la dispersion de las mediciones. Las configuraciones de uno a cuatro hilos tienen desviaciones estandar minimas; las de seis y ocho muestran valores extremos aislados, que es lo que se espera cuando el planificador del sistema operativo decide, corrida a corrida, en que tipo de nucleo colocar cada hilo. Es tambien la razon por la que el informe reporta promedio, minimo, maximo y desviacion, y no solo el promedio.

8.3 El tamano del problema decide si vale la pena paralelizar

Con $N = 250$ la version ajustada con ocho hilos alcanza un speedup de 1.31x, es decir que practicamente no gana nada. El motivo es que un paso secuencial cuesta solo 0.411 ms, y crear el equipo de hilos, repartir el trabajo y sincronizar en la barrera cuesta un orden de magnitud comparable. La regla practica que se desprende de las mediciones es que la paralelizacion empieza a rendir cuando el trabajo por paso supera aproximadamente el milisegundo, lo que en este programa ocurre entre $N = 500$ y $N = 1\,000$.

Ese comportamiento es la ilustracion directa de la ley de Gustafson: la fraccion secuencial del paso —preparar punteros, abrir la region, fusionar contadores— es casi constante, mientras que la fraccion paralela crece con N^2 . Al aumentar N la fraccion secuencial se vuelve despreciable y la eficiencia mejora, que es exactamente lo que muestra el mapa de calor de la Figura 3: la region oscura se ensancha hacia la derecha conforme se baja en la tabla.

8.4 Costo y beneficio de cada iteracion

Vale la pena separar cuanto aporte cada una de las tres mejoras que distinguen la version ajustada de la estatica, porque no todas rindieron lo mismo:

- **Region paralela unica.** Ahorra la creacion de un equipo de hilos por sub-paso. Con dos sub-pasos por cuadro el ahorro es real pero modesto, y solo se nota con N pequeno, donde la sobrecarga pesa.

- **Reparto guiado.** Es la mejora que produjo casi toda la ganancia medible, pero unicamente con ocho hilos y carga alta, que es el caso en el que dos nucleos lentos se suman al equipo. Con cuatro hilos las dos versiones empatan, y con seis la estatica es incluso mejor, porque ahi no hay desbalance que corregir y el reparto guiado solo agrega costo de planificacion.
- **Contadores privatizados.** No mejoro el tiempo de forma medible en este programa, porque la cantidad de pelotas que llegan al fondo en un paso es pequena frente a N y la contencion sobre las operaciones atómicas era baja. Se conserva porque es la solución correcta y porque su ventaja crecería si la pirámide fuera mas corto o las pelotas cayeran mas rapido.

Reportar la tercera mejora como neutra es tan util como reportar las otras dos: confirma que la contencion estaba en el reparto de la carga y no en los contadores, que era la hipótesis inicial del equipo.

8.5 Limitaciones del estudio

- Todas las mediciones provienen de una sola maquina. Un procesador con nucleos homogeneos debería mostrar una curva de eficiencia mas plana hasta el total de nucleos, y probablemente una diferencia menor entre reparto estatico y guiado.
- El sistema operativo no estaba aislado: aunque no se ejecutaron tareas pesadas en paralelo, macOS mantiene procesos de fondo que compiten por los nucleos. Es una de las causas de la dispersion observada con seis y ocho hilos.
- macOS no permite fijar hilos a nucleos concretos, de modo que no fue posible probar politicas de afinidad (`OMP_PROC_BIND`) que en Linux habrian permitido separar el efecto del hardware del efecto del planificador.
- El banco de pruebas mide solo la fisica. El costo del dibujado se midio aparte —el HUD reporta ms de fisica y ms por cuadro por separado— pero no se paralelizo, porque el contexto de OpenGL no es seguro para multiples hilos.

9. Conclusiones

- La paralelización con OpenMP aceleró el núcleo de simulación en todos los tamaños probados alcanzando un speedup máximo de **5.15x** con 8 hilos y $N = 4\,000$ pelotas (eficiencia 0.64), y una eficiencia superior a 0.85 de forma sostenida con dos y cuatro hilos siempre que N sea mayor o igual a 2 000.
- Asignar un hilo del sistema operativo por cada elemento de la simulación es una estrategia perdedora: resultó más de cinco veces más lenta que la versión secuencial y dejó de ser viable por encima de 1 024 elementos. El costo de coordinar hilos crece con la cantidad de tareas mientras el trabajo por tarea permanece constante.
- La etapa de aglomeración del método PCAM no es un trámite: es exactamente la etapa que separa la versión que funciona de la que no. Agrupar muchas pelotas en un bloque y darle ese bloque a un hilo reutilizado es lo único que cambia entre ambas.
- Diseñar el estado con doble buffer —un arreglo de solo lectura y otro con un único escritor por índice— elimina las condiciones de carrera por construcción, sin costo de sincronización, y además hace la simulación determinista. Ese determinismo es lo que permitió verificar la corrección con pruebas automatizadas que comparan bit a bit el resultado de los cuatro modos.
- La elección de la política de reparto importa más que la cantidad de directivas, pero no existe una que gane siempre. En un procesador con núcleos asimétricos, `schedule(guided)` recuperó una parte sustancial del rendimiento que `schedule(static)` perdía por desbalance cuando el equipo incluía los dos núcleos lentos y había trabajo suficiente que repartir; en cambio, con cargas pequeñas el reparto estático ganó en todas las cantidades de hilos, porque ahí el costo de planificación del reparto guiado no alcanza a amortizarse.
- El beneficio de paralelizar depende del tamaño del problema. Por debajo de unas 500 pelotas la sobrecarga de la región paralela consume la ganancia; a partir de ahí la eficiencia crece de forma sostenida con N , tal como describe la ley de Gustafson.
- En términos del objetivo del screensaver, la paralelización permitió duplicar con holgura la cantidad de elementos que se pueden simular manteniendo 60 cuadros por segundo: la versión secuencial pierde el presupuesto entre 2 000 y 4 000 pelotas, mientras que la versión ajustada con ocho hilos todavía lo cumple con 4 000.

10. Recomendaciones

- **Reducir la complejidad antes de paralelizar más.** El siguiente salto de rendimiento no vendrá de más hilos sino de un mejor algoritmo: una rejilla espacial uniforme reduciría las colisiones de $O(N^2)$ a $O(N)$ esperado. Paralelizar un algoritmo cuadrático da un factor constante; cambiar el algoritmo cambia la pendiente.

- **Repetir las mediciones en hardware homogéneo.** Buena parte del análisis de este informe gira en torno a la asimetría del M1 Pro. Medir en un procesador con núcleos iguales permitiría separar lo que es propio del programa de lo que es propio de la máquina.
- **Explorar políticas de afinidad.** En un sistema que lo permita, `OMP_PROC_BIND` y `OMP_PLACES` permitirían restringir el equipo a los núcleos de rendimiento y comprobar la hipótesis de la sección 8.2 de forma directa.
- **Ajustar el tamaño de bloque.** Se probaron los repartos estático y guiado con sus valores por omisión. Un barrido de `schedule(dynamic, k)` para distintos `k` podría encontrar un punto intermedio entre el costo de planificación y el balance de carga.
- **No paralelizar el dibujado, sino reducirlo.** El contexto de OpenGL no admite múltiples hilos, pero el renderizado ya se resolvió con un cuadrilátero texturizado por elemento en lugar de una malla de esfera. Si el dibujado volviera a ser el cuello de botella, el camino es el instanciado o un buffer de vértices, no más hilos.
- **Conservar la versión lenta en el repositorio.** Mantener la implementación de un hilo por pelota junto a las de OpenMP tiene valor didáctico y experimental: es el contraejemplo que da sentido a las demás y permite reproducir la comparación en cualquier máquina.

11. Bibliografía

Las referencias siguen el formato APA. Se listan únicamente las fuentes efectivamente consultadas durante el desarrollo del proyecto.

OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*.

OpenMP ARB. Recuperado de <https://www.openmp.org/specifications/>

Chapman, B., Jost, G., y van der Pas, R. (2007). *Using OpenMP: Portable Shared Memory Parallel Programming*. Cambridge, MA: The MIT Press.

Pacheco, P. S., y Malensek, M. (2022). *An Introduction to Parallel Programming* (2a ed.). Cambridge, MA: Morgan Kaufmann.

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En *Proceedings of the AFIPS Spring Joint Computer Conference* (pp. 483-485). Nueva York: ACM.

Gustafson, J. L. (1988). Reevaluating Amdahl's Law. *Communications of the ACM*, 31(5), 532-533.

Foster, I. (1995). *Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering*. Reading, MA: Addison-Wesley.

SDL Community. (2024). *SDL2 Wiki: API Reference*. Simple DirectMedia Layer. Recuperado de <https://wiki.libsdl.org/SDL2/>

Apple Inc. (2021). *Apple M1 Pro and M1 Max: Technical Overview*. Apple Newsroom. Recuperado de <https://www.apple.com/newsroom/>

Anexo 1. Diagrama de flujo del programa

El diagrama recorre el programa completo, desde la captura de argumentos hasta la liberacion de recursos. Se presenta en dos partes unidas por el conector **A**. Las formas siguen la convencion habitual: ovalo para inicio y fin, rectangulo para proceso, rombo para decision, paralelogramo para entrada y salida, y rectangulo con doble borde para las secciones que se ejecutan en paralelo. Los recuadros del grupo 7 indican de forma explicita el mecanismo de sincronia que usa cada version.

El archivo fuente del diagrama esta en `docs/graficas/diagrama_flujo.dot` y hay tambien una version vectorial en PDF, que permite ampliar sin perdida.

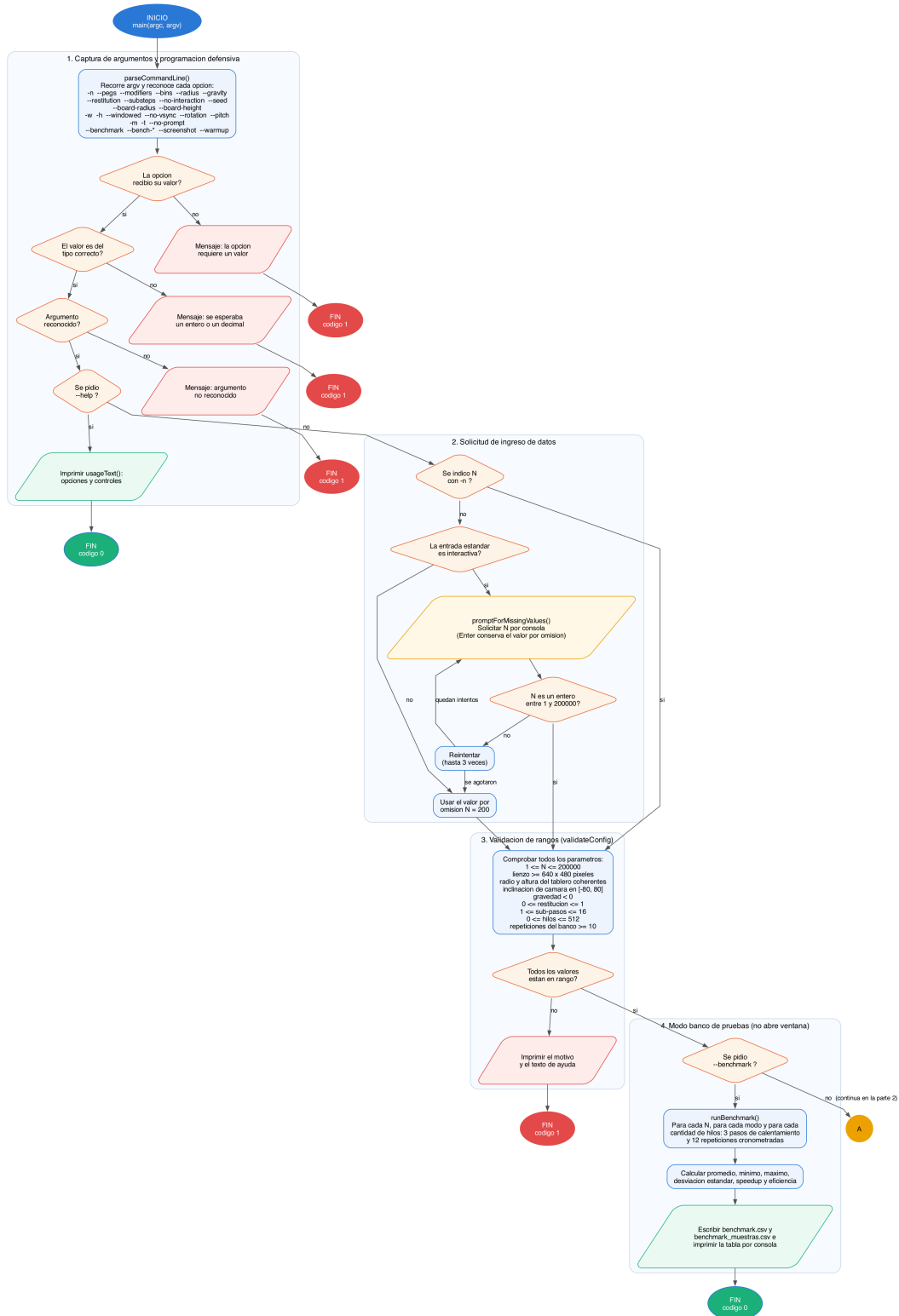


Figura 8. Diagrama de flujo, parte 1 de 2: arranque, captura de argumentos con programacion defensiva, solicitud de ingreso de datos, validacion de rangos y modo de banco de pruebas.

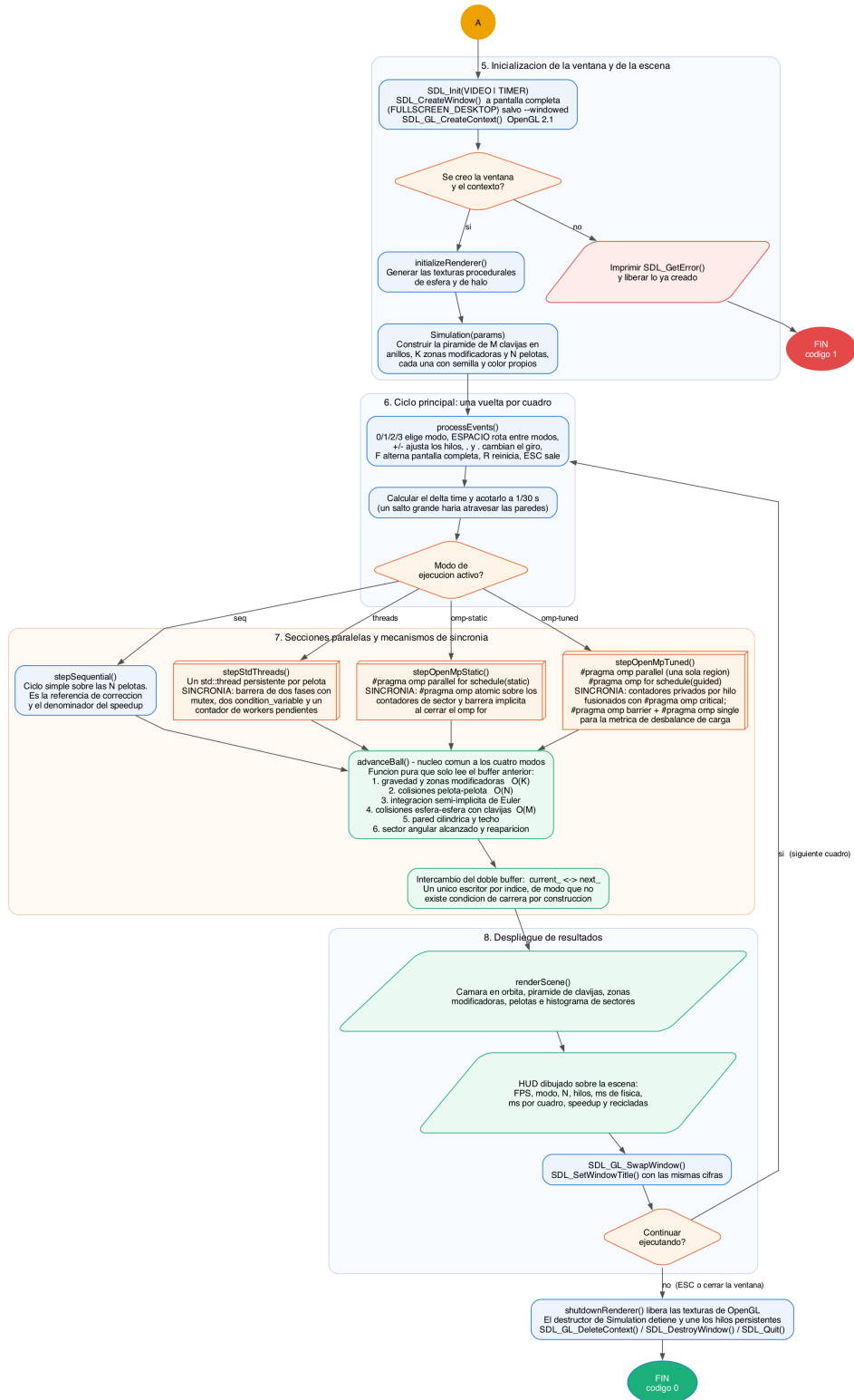


Figura 9. Diagrama de flujo, parte 2 de 2: creacion de la ventana y de la escena, ciclo principal, secciones paralelas con sus mecanismos de sincronia, despliegue de resultados y liberacion ordenada de recursos.

Anexo 2. Catalogo de funciones

El catalogo cubre las funciones, clases y estructuras relevantes del proyecto, agrupadas por archivo. Para cada una se indican sus entradas con nombre y tipo, sus salidas y una descripcion de su proposito y de su funcionamiento. Se incluyen tambien las funciones internas de cada unidad de traduccion cuando resultan necesarias para entender el diseno.

include/Vec3.h

Funcion / tipo	Entradas	Salidas	Descripcion
struct Vec3	x, y, z : float — componentes de ancho, alto y profundidad del tablero.	No aplica (tipo de dato).	Vector de tres componentes definido como agregado POD para que el arreglo de pelotas quede contiguo en memoria y los ciclos puedan repartirse entre hilos sin direcciones.
Vec3& operator+=(const Vec3& other)	other : const Vec3& — vector que se suma.	Vec3& — referencia al propio vector ya modificado.	Suma componente a componente en el lugar. Se usa al integrar la posicion y al acumular impulsos.
Vec3 operator*(const Vec3& vector, float scalar)	vector : const Vec3& — vector a escalar; scalar : float — factor.	Vec3 — vector escalado.	Multiplicacion por escalar. Aparece en cada paso de integracion (velocidad por delta time).
float dot(const Vec3& left, const Vec3& right)	left, right : const Vec3& — vectores a proyectar.	float — producto punto.	Producto punto. Base de las proyecciones que resuelven los rebotes: proyecta la velocidad relativa sobre la normal de contacto.
float lengthSquared(const Vec3& vector)	vector : const Vec3& — vector medido.	float — magnitud al cuadrado.	Magnitud al cuadrado. Se prefiere sobre length() dentro del ciclo $O(N^2)$ porque evita la raiz cuadrada cuando solo hay que comparar distancias.
float length(const Vec3& vector)	vector : const Vec3& — vector medido.	float — magnitud euclidiana.	Magnitud del vector. Solo se invoca cuando la distancia real es necesaria.
Vec3 normalized(const Vec3& vector)	vector : const Vec3& — vector a normalizar.	Vec3 — vector unitario; si la magnitud es despreciable devuelve (0,1,0).	Normalizacion con proteccion contra division entre cero: ante un vector nulo devuelve un eje estable en lugar de producir NaN (programacion defensiva).

include/Random.h

Funcion / tipo	Entradas	Salidas	Descripcion
std::uint32_t nextRandomUInt(std::uint32_t& state)	state : std::uint32_t& — semilla, se modifica en el lugar.	std::uint32_t — entero pseudoaleatorio de 32 bits.	Generador SplitMix32. Cada pelota lleva su propia semilla, de modo que la secuencia que consume no depende del numero de hilos ni de su calendarizacion y no requiere candados.
float nextRandomFloat(std::uint32_t& state)	state : std::uint32_t& — semilla del generador.	float — valor uniforme en [0, 1).	Convierte 24 bits del generador en un flotante. Se usa para colores y decisiones de escena.

Funcion / tipo	Entradas	Salidas	Descripcion
<code>float nextRandomInRange(std::uint32_t& state, float minimum, float maximum)</code>	<code>state : std::uint32_t& — semilla; minimum, maximum : float — extremos del intervalo.</code>	<code>float — valor uniforme en [minimum, maximum).</code>	Escala el valor uniforme al rango pedido. Genera posiciones y velocidades de reaparicion.
<code>std::uint32_t mixSeed(std::uint32_t seed, std::uint32_t index)</code>	<code>seed : std::uint32_t — semilla global; index : std::uint32_t — indice de la pelota.</code>	<code>std::uint32_t — semilla derivada.</code>	Deriva la semilla de cada pelota a partir de la global y de su indice sin recorrer el generador secuencialmente, lo que permite inicializar en paralelo de forma reproducible.

include/Entities.h

Funcion / tipo	Entradas	Salidas	Descripcion
<code>struct Ball</code>	<code>position, velocity, color : Vec3; radius, mass : float; seed : std::uint32_t; bin : std::int32_t; active : bool.</code>	No aplica (tipo de dato).	Pelota del tablero. Es la unidad de trabajo que se reparte entre hilos: el arreglo de Ball es el unico que crece con N.
<code>struct Peg</code>	<code>basePosition, color : Vec3; radius, amplitude, angularSpeed, phase : float; level : std::int32_t.</code>	No aplica (tipo de dato).	Clavija de la piramide. El campo level indica su nivel, con 0 en el vertice, y determina su color. Una parte de las clavijas oscila radialmente para que el recorrido de las pelotas no sea repetitivo.
<code>struct Modifier</code>	<code>center, color : Vec3; radius, strength : float; kind : ModifierKind.</code>	No aplica (tipo de dato).	Zona esferica que altera la fisica local: pozo gravitatorio, impulso de velocidad o turbulencia lateral dependiente del tiempo.
<code>Vec3 pegPositionAt(const Peg& peg, float time)</code>	<code>peg : const Peg& — clavija consultada; time : float — reloj de simulacion en segundos.</code>	<code>Vec3 — centro de la clavija desplazado por su oscilacion.</code>	Evalua la posicion de la clavija en el instante dado mediante una funcion seno. La oscilacion es radial: la clavija se acerca y se aleja del eje de la piramide, y la del vertice, que esta sobre el eje, oscila verticalmente. Es una funcion pura: cualquier hilo puede llamarla a la vez sin sincronizacion.

include/SimulationConfig.h

Funcion / tipo	Entradas	Salidas	Descripcion
<code>enum class ExecutionMode</code>	<code>Sequential, StdThreads, OpenMpStatic, OpenMpTuned.</code>	No aplica (tipo de dato).	Identifica cual de las cuatro estrategias de actualizacion se ejecuta.
<code>struct SimulationParams</code>	<code>ballCount, pegLevels, pegsPerBaseRing, modifierCount, binCount, substeps : int; ballRadius, gravity, restitution, boardRadius, boardHeight : float; ballInteraction : bool; seed : std::uint32_t.</code>	<code>floorY(), ceilingY(), usableRadius(), pyramidRadius() : float — limites derivados.</code>	Agrupar todos los parametros fisicos y de escena. El banco de pruebas los reutiliza sin necesidad de abrir una ventana.

Funcion / tipo	Entradas	Salidas	Descripcion
struct AppConfig	simulation : SimulationParams; mode : ExecutionMode; windowWidth, windowHeight, threadCount, benchmarkRepetitions, benchmarkSteps, screenshotWarmupFrames : int; vsync, fullscreen, showHelp, runBenchmark, allowPrompt : bool; rotationSpeed, cameraPitch : float; benchmarkBallCounts, benchmarkThreadCounts : std::vector; benchmarkOutput, screenshotPath : std::string.	No aplica (tipo de dato).	Configuracion completa de la aplicacion tal como queda despues de leer la linea de comandos y de validarla.

src/SimulationConfig.cpp

Funcion / tipo	Entradas	Salidas	Descripcion
bool parseInteger(const std::string& text, int& value)	text : const std::string& — cadena recibida en la linea de comandos.	value : int& — entero convertido. Retorno bool: true si la conversion fue total.	Convierte a entero exigiendo que se consuma toda la cadena y detectando desbordamiento. Rechaza entradas como '12abc' que strtol aceptaria parcialmente.
bool parseFloatValue(const std::string& text, float& value)	text : const std::string& — cadena recibida.	value : float& — decimal convertido. Retorno bool: true si la conversion fue total.	Equivalente de parseInteger para valores decimales como la gravedad o la restitution.
bool parseIntegerList(const std::string& text, std::vector& values)	text : const std::string& — lista separada por comas, por ejemplo '250,500,1000'.	values : std::vector& — enteros positivos. Retorno bool: true si todos son validos.	Analiza las listas de --bench-balls y --bench-threads, recortando espacios y exigiendo que cada elemento sea un entero positivo.

include/SimulationConfig.h

Funcion / tipo	Entradas	Salidas	Descripcion
const char* executionModeName(ExecutionMode mode)	mode : ExecutionMode — modo consultado.	const char* — nombre corto del modo.	Devuelve la etiqueta que aparece en el HUD, en el titulo de la ventana y en los CSV.
bool parseExecutionMode(const std::string& text, ExecutionMode& mode)	text : const std::string& — cadena escrita por el usuario.	mode : ExecutionMode& — modo reconocido. Retorno bool: false si la cadena no corresponde.	Acepta varios alias por modo (seq/secuencial, omp-tuned/tuned/omp2) y rechaza el resto.
std::string usageText(const char* programName)	programName : const char* — nombre con el que se invoco el ejecutable.	std::string — texto de ayuda completo.	Construye la ayuda que se imprime con --help y tambien despues de cualquier error de argumentos, para que el usuario vea de inmediato la forma correcta de invocacion.

Funcion / tipo	Entradas	Salidas	Descripcion
<code>bool parseCommandLine(int argc, char** argv, AppConfig& config, std::string& errorMessage)</code>	<code>argc : int, argv : char**</code> — argumentos tal como los recibe main.	<code>config : AppConfig&</code> — configuracion resultante; <code>errorMessage : std::string&</code> — descripcion del primer problema. Retorno <code>bool</code> : true si los argumentos son validos.	Recorre argv reconociendo cada opcion, incluidas las de camara (<code>--rotation</code> , <code>--pitch</code>), ventana (<code>--windowed</code>) y geometria (<code>--board-radius</code> , <code>--board-height</code>), verifica que las que llevan valor lo reciban, convierte los tipos, invoca la solicitud interactiva si falta N y termina llamando a <code>validateConfig</code> . Es el punto unico de entrada de la programacion defensiva.
<code>bool validateConfig(const AppConfig& config, std::string& errorMessage)</code>	<code>config : const AppConfig&</code> — configuracion a revisar.	<code>errorMessage : std::string&</code> — motivo del rechazo. Retorno <code>bool</code> : true si todo esta en rango.	Comprueba N, la rejilla de clavijas, modificadores, casillas, sub-pasos, radio, gravedad, restitution, tamano minimo del lienzo (640x480), cantidad de hilos y los parametros del banco de pruebas.
<code>void promptForMissingValues(AppConfig& config, bool ballCountProvided)</code>	<code>config : AppConfig&</code> — configuracion en construccion; <code>ballCountProvided : bool</code> — si el usuario ya indico N con -n.	<code>config : AppConfig&</code> — se completa con el valor ingresado.	Solicita N por consola cuando no vino por argumento. Solo actua si la entrada estandar es interactiva, admite Enter para conservar el valor por omision y reintenta hasta tres veces ante entradas invalidas.

include/Simulation.h

Funcion / tipo	Entradas	Salidas	Descripcion
<code>Ball advanceBall(std::size_t index, const Ball* balls, std::size_t ballCount, const Peg* pegs, std::size_t pegCount, const Modifier* modifiers, std::size_t modifierCount, const SimulationParams& params, float time, float dt, int& binHit)</code>	<code>index : std::size_t</code> — pelota a actualizar; <code>balls, pegs, modifiers</code> : punteros de solo lectura al estado del cuadro anterior; <code>ballCount, pegCount, modifierCount</code> : <code>std::size_t</code> ; <code>params : const SimulationParams&</code> ; <code>time : float</code> — reloj de simulacion; <code>dt : float</code> — paso.	<code>binHit : int&</code> — casilla alcanzada, o -1 si la pelota no llego al fondo. Retorno <code>Ball</code> : el nuevo estado de la pelota.	NUCLEO DE LA PARALELIZACION. Aplica gravedad y zonas modificadoras O(K), colisiones pelota-pelota O(N), integracion semi-implicita de Euler, colisiones esfera-esfera contra las clavijas de la piramide O(M), rebote contra la pared cilindrica y el techo, y anotacion del sector angular alcanzado con reaparicion arriba. Es una funcion pura sobre entradas de solo lectura, de modo que varios hilos pueden ejecutarla simultaneamente sin candados y el resultado no depende del orden de ejecucion.
<code>Simulation::Simulation(const SimulationParams& params)</code>	<code>params : const SimulationParams&</code> — parametros ya validados.	No aplica (constructor).	Construye la escena completa invocando <code>reset()</code> con la semilla indicada.
<code>Simulation::~Simulation()</code>	No aplica.	No aplica (destructor).	Libera el sistema de hilos persistentes, que a su vez los detiene y los une ordenadamente antes de destruir cualquier memoria que esten leyendo.
<code>void Simulation::reset(std::uint32_t seed)</code>	<code>seed : std::uint32_t</code> — semilla global del generador.	Modifica el estado interno: clavijas, modificadores, pelotas y contadores.	Regenera la escena completa de forma reproducible. Tambien libera los hilos persistentes, porque su cantidad depende de N.
<code>void Simulation::resizeBallCount(int ballCount)</code>	<code>ballCount : int</code> — nuevo valor de N (se recorta al rango valido).	Modifica el estado interno.	Cambia la cantidad de pelotas conservando la semilla, para poder variar la carga sin reiniciar el programa.

Funcion / tipo	Entradas	Salidas	Descripcion
<code>void Simulation::step(float dt, ExecutionMode mode, int threadCount)</code>	<code>dt : float</code> — segundos a avanzar; <code>mode : ExecutionMode</code> — estrategia; <code>threadCount : int</code> — hilos de OpenMP (0 usa todos los disponibles).	Modifica el estado interno de la simulacion.	Despachador: encamina el paso a una de las cuatro implementaciones.
<code>void Simulation::stepSequential(float dt)</code>	<code>dt : float</code> — segundos a avanzar.	Modifica <code>current_</code> , <code>next_</code> , <code>binCounts_</code> y <code>recycledBalls_</code> .	Version secuencial de referencia. Recorre las N pelotas en un solo hilo y acumula las casillas en el mismo ciclo. Es el denominador de todos los speedup reportados.
<code>void Simulation::stepStdThreads(float dt)</code>	<code>dt : float</code> — segundos a avanzar.	Modifica el estado interno; usa <code>binHits_</code> como area de escritura por indice.	Primera version paralela: delega cada pelota a un <code>std::thread</code> persistente y espera en una barrera de dos fases. Si el sistema operativo no permite crear los hilos, degrada de forma controlada a la version secuencial en lugar de abortar.
<code>void Simulation::stepOpenMpsTatic(float dt, int threadCount)</code>	<code>dt : float</code> — segundos a avanzar; <code>threadCount : int</code> — hilos solicitados.	Modifica el estado interno.	Segunda version paralela: <code>'#pragma omp parallel for schedule(static)'</code> sobre el arreglo de pelotas, con <code>'#pragma omp atomic'</code> sobre los contadores de casilla y <code>'reduction(+)'</code> sobre el total de recicladas. La barrera implicita al cerrar el <code>omp for</code> es la sincronia que necesita el sub-paso siguiente.
<code>void Simulation::stepOpenMptUned(float dt, int threadCount)</code>	<code>dt : float</code> — segundos a avanzar; <code>threadCount : int</code> — hilos solicitados.	Modifica el estado interno y actualiza <code>lastLoadImbalance_</code> .	Tercera version paralela: abre una sola region paralela para todos los sub-pasos, reparte con <code>'schedule(guided)'</code> , acumula las casillas en contadores privados por hilo que se fusionan con <code>'#pragma omp critical'</code> , y calcula el desbalance de carga con <code>'#pragma omp barrier'</code> seguido de <code>'#pragma omp single'</code> . La alternancia de buffers se resuelve por la paridad del sub-paso, porque dentro de la region no se puede intercambiar los vectores.

src/Simulation.cpp

Funcion / tipo	Entradas	Salidas	Descripcion
<code>void Simulation::buildPegs(std::uint32_t& seedState)</code>	<code>seedState : std::uint32_t&</code> — generador de la escena.	Llena <code>pegs_</code> .	Construye la piramide de clavijas como una pila de anillos: el nivel 0 es un vertice de una sola clavija y cada nivel siguiente crece en radio y en cantidad, de modo que la separacion entre clavijas vecinas se mantiene constante. Los niveles impares se giran medio paso angular para que no queden canales rectos por los que las pelotas caigan sin rebotar. Alrededor del 30 % de las clavijas oscila y se pinta enambar; el resto sigue una rampa fria de cian a azul segun el nivel.

Funcion / tipo	Entradas	Salidas	Descripcion
<code>void Simulation::buildModifiers(std::uint32_t& seedState)</code>	<code>seedState : std::uint32_t&</code> — generador de la escena.	Llena <code>modifiers_</code> .	Coloca K zonas de influencia con centro, radio, tipo e intensidad pseudoaleatorios.
<code>void Simulation::spawnBall(Ball& ball, std::uint32_t index, std::uint32_t& seedState) const</code>	<code>ball : Ball&</code> — pelota a inicializar; <code>index : std::uint32_t</code> — su indice; <code>seedState : std::uint32_t&</code> — generador de la escena.	<code>ball : Ball&</code> — queda con semilla, radio, masa, color, posicion y velocidad.	Inicializa una pelota derivando su semilla propia con <code>mixSeed</code> y repartandola por toda la altura y todo el radio del cilindro, para que la escena se vea poblada desde el primer cuadro en lugar de arrancar con una sola columna cayendo sobre el vertice.
<code>void Simulation::ensureStdThreads()</code>	No aplica.	Crea o reemplaza <code>stdThreads_</code> ; en caso de fallo llena <code>stdThreadsError_</code> .	Crea el sistema de un hilo por pelota solo cuando hace falta y captura la excepcion si el sistema operativo rechaza la creacion, marcando el modo como no disponible.
<code>Vec3 hsvToRgb(float hue, float saturation, float value)</code>	<code>hue : float</code> en [0,1]; <code>saturation, value : float</code> en [0,1].	<code>Vec3</code> — componentes rojo, verde y azul en [0,1].	Convierte HSV a RGB. Sortear solo el tono, con saturacion y valor altos fijos, produce colores vistosos y evita los tonos apagados que daría sortear las tres componentes RGB por separado.
<code>void respawnAtTop(Ball& ball, const SimulationParams& params)</code>	<code>ball : Ball&</code> — pelota reciclada; <code>params : const SimulationParams&</code> — limites del tablero.	<code>ball : Ball&</code> — con posicion, velocidad y color nuevos.	Recoloca la pelota en un disco estrecho sobre el vertice de la piramide, con la altura escalonada para que las pelotas recicladas no se amontonen en una columna. Consume unicamente el generador propio de la pelota, por lo que puede ejecutarse dentro de un ciclo paralelo sin sincronizacion.
<code>void clampSpeed(Vec3& velocity)</code>	<code>velocity : Vec3&</code> — velocidad a limitar.	<code>velocity : Vec3&</code> — con magnitud recortada al maximo permitido.	Limita la rapidez sin alterar la direccion, para que un choque multiple no dispare una pelota fuera del tablero.

include/BallThreadSystem.h

Funcion / tipo	Entradas	Salidas	Descripcion
<code>BallThreadSystem::BallThreadSystem(std::size_t ballCount)</code>	<code>ballCount : std::size_t</code> — cantidad de hilos a crear (uno por pelota).	No aplica (constructor). Lanza <code>std::system_error</code> si el sistema operativo los rechaza.	Crea los hilos persistentes una sola vez. Si falla a mitad de camino, detiene y une los que ya existen antes de propagar la excepcion, de modo que no quedan hilos huérfanos.
<code>BallThreadSystem::~~BallThreadSystem()</code>	No aplica.	No aplica (destructor).	Levanta la bandera de parada bajo el mutex, despierta a todos los workers y los une. Ningun hilo sobrevive al objeto.
<code>void BallThreadSystem::runRound(const Ball* readBalls, Ball* writeBalls, std::size_t ballCount, const Peg* pegs, std::size_t pegCount, const Modifier* modifiers, std::size_t modifierCount, const SimulationParams& params, float time, float dt, int* binHits)</code>	<code>readBalls, pegs, modifiers :</code> punteros de solo lectura al cuadro anterior; <code>ballCount, pegCount, modifierCount :</code> <code>std::size_t</code> ; <code>params : const SimulationParams&</code> ; <code>time, dt :</code> <code>float</code> .	<code>writeBalls : Ball*</code> — estado nuevo; <code>binHits : int*</code> — casilla alcanzada por cada pelota.	Ejecuta una ronda completa. Primera fase de la barrera: publica los datos, incrementa el numero de generacion y despierta a todos los workers. Segunda fase: espera a que el contador de pendientes llegue a cero, lo que garantiza que nadie lea <code>writeBalls</code> mientras otro hilo todavía lo escribe.

src/BallThreadSystem.cpp

Funcion / tipo	Entradas	Salidas	Descripcion
void BallThreadSystem::workerLoop(std::size_t ballIndex)	ballIndex : std::size_t — indice fijo de la pelota que atiende este hilo.	Escribe writeBalls[ballIndex] y binHits[ballIndex].	Ciclo de vida de cada worker: espera su turno comparando el numero de generacion (lo que descarta los despertares espurios), copia los datos de la ronda, suelta el mutex, calcula su pelota fuera de la seccion critica y decrementa el contador de pendientes.

include/Renderer.h

Funcion / tipo	Entradas	Salidas	Descripcion
struct HudInfo	framesPerSecond, physicsMilliseconds, frameMilliseconds, speedupEstimate, loadImbalance : double; threads, ballCount : int; recycled : long long; modeName, notice : std::string.	No aplica (tipo de dato).	Cifras que el HUD despliega sobre la escena.
void initializeRenderer(int width, int height)	width, height : int — tamano del lienzo en pixeles.	Modifica el estado de OpenGL y crea dos texturas.	Prepara profundidad, mezcla y prueba de alfa, y genera por codigo las texturas de esfera iluminada y de halo. No lee ningun archivo externo, de modo que el proyecto se entrega solo como codigo fuente.
void shutdownRenderer()	No aplica.	Libera las texturas de OpenGL.	Destruccion explicita de los recursos graficos antes de eliminar el contexto.
void resizeRenderer(int width, int height)	width, height : int — nuevo tamano del lienzo.	Modifica el viewport y la matriz de proyeccion.	Recalcula la perspectiva cuando el usuario cambia el tamano de la ventana.
void renderScene(const Simulation& simulation, const HudInfo& hud, const CameraState& camera)	simulation : const Simulation& — estado de solo lectura; hud : const HudInfo& — cifras; camera : const CameraState& — orientacion de la camara.	Dibuja el cuadro completo en el framebuffer activo.	Dibuja el piso, la corona de sectores con su histograma, el aro del cilindro, las aristas y los anillos de la piramide, las zonas modificadoras, las clavijas, el halo aditivo, las pelotas y el HUD. Coloca la camara en orbita segun el estado recibido. Se invoca solo despues de que la fisica termino y la barrera libero a todos los hilos.
void drawText(const std::string& text, float x, float y, float scale, float r, float g, float b, float a)	text : const std::string& — cadena; x, y : float — posicion en pixeles desde la esquina superior izquierda; scale : float — tamano del pixel del tipo de letra; r, g, b, a : float — color.	Dibuja cuadrilateros en el framebuffer.	Dibuja texto con el tipo de letra de mapa de bits 5x7 incluido en el programa. Todos los glifos de una llamada se emiten dentro de un solo par glBegin/glEnd.

src/Renderer.cpp

Funcion / tipo	Entradas	Salidas	Descripcion
<code>GLuint createSphereTexture()</code>	No aplica.	GLuint — identificador de la textura creada.	Genera por codigo una esfera preiluminada: reconstruye la normal a partir de la posicion dentro del circulo y evalua un termino difuso mas un realce especular. Multiplicada por el color de la pelota, produce una esfera creible con un solo cuadrilatero.
<code>float cameraDistanceFor(const SimulationParams& params, float aspect)</code>	<code>params : const SimulationParams& — dimensiones del tablero;</code> <code>aspect : float — relacion de aspecto del lienzo.</code>	float — distancia de la camara sobre el eje Z.	Calcula a que distancia colocar la camara para que la piramide completa quepa sin importar la relacion de aspecto de la ventana. Suma el radio del cilindro porque la cara cercana avanza hacia la camara.
<code>void emitBillboard(float x, float y, float z, float radius)</code>	<code>x, y, z : float — centro;</code> <code>radius : float — semilado del cuadrilatero.</code>	Emite cuatro vertices texturizados.	Dibuja un cuadrilatero orientado a la camara, construido sobre los ejes de pantalla llevados a coordenadas de mundo. Como la camara orbita, un cuadrilatero fijo en el plano XY dejaria de mirarla en cuanto girara.

include/Benchmark.h

Funcion / tipo	Entradas	Salidas	Descripcion
<code>struct BenchmarkRecord</code>	<code>ballCount, threads, steps, repetitions : int; mode : ExecutionMode;</code> <code>averageStepMs, minimumStepMs, maximumStepMs, stdDevStepMs, speedupAverage, speedupBest, efficiency, maxFps : double; available : bool; note : std::string; samples : std::vector.</code>	No aplica (tipo de dato).	Resultado agregado de una combinacion de N, modo y cantidad de hilos, junto con las muestras individuales de cada repeticion.
<code>int runBenchmark(const AppConfig& config)</code>	<code>config : const AppConfig& — configuracion validada con runBenchmark activo.</code>	Escribe benchmark.csv y benchmark_muestras.csv; imprime la tabla por consola. Retorno int: 0 si todo fue bien.	Recorre todas las combinaciones de N, modo y hilos; para cada una ejecuta pasos de calentamiento y luego las repeticiones cronometradas, calcula las estadisticas y los speedup respecto de la version secuencial con el mismo N, y guarda los resultados.

src/Benchmark.cpp

Funcion / tipo	Entradas	Salidas	Descripcion
<code>BenchmarkRecord measure(const SimulationParams& params, ExecutionMode mode, int threads, int repetitions, int steps)</code>	<code>params : const SimulationParams&; mode : ExecutionMode; threads, repetitions, steps : int.</code>	BenchmarkRecord — estadisticas y muestras individuales.	Mide una combinacion concreta. Antes de cronometrar ejecuta tres pasos de calentamiento para llenar las caches y para que OpenMP cree su equipo de hilos, de modo que ese costo no contamine la primera toma. Usa un paso de tiempo fijo de 1/60 s para eliminar el ruido del reloj real.

Funcion / tipo	Entradas	Salidas	Descripcion
int stepsForBallCount(int ballCount, int requested, int substeps)	ballCount : int — N evaluado; requested : int — tope indicado por el usuario; substeps : int — sub-pasos por cuadro.	int — pasos a cronometrar, siempre en el rango [4, requested].	Reduce la cantidad de pasos de forma inversamente proporcional a N^2 , que es la complejidad del nucleo, para que N grande no dispare el tiempo total del banco.
void ensureParentDirectory(const std::string& path)	path : const std::string& — ruta del archivo de salida.	Crea los directorios intermedios que falten.	Programacion defensiva: evita que el banco termine sin poder guardar el CSV despues de varios minutos de medicion.

include/Font5x7.h

Funcion / tipo	Entradas	Salidas	Descripcion
const std::uint8_t* glyphFor(char character)	character : char — caracter buscado.	const std::uint8_t* — cinco bytes, uno por columna del glifo.	Devuelve el glifo del tipo de letra 5x7, convirtiendo minusculas a mayusculas y sustituyendo por espacio cualquier caracter fuera de la tabla.

src/main.cpp

Funcion / tipo	Entradas	Salidas	Descripcion
int main(int argc, char** argv)	argc : int, argv : char** — argumentos de la linea de comandos.	int — 0 si la ejecucion fue correcta, 1 ante cualquier error.	Punto de entrada. Captura y valida argumentos, decide entre banco de pruebas y screensaver, crea la ventana y el contexto, ejecuta el ciclo principal y libera todos los recursos en orden inverso a su creacion.
void processEvents(RuntimeState& state)	state : RuntimeState& — estado del ciclo principal.	state : RuntimeState& — modificado segun las teclas pulsadas.	Procesa la cola de SDL: cierre de ventana, cambio de tamano, seleccion de modo con 0/1/2/3, rotacion con ESPACIO, ajuste de hilos con + y -, reinicio con R y salida con ESC.
ExecutionMode nextMode(ExecutionMode mode)	mode : ExecutionMode — modo actual.	ExecutionMode — siguiente modo del ciclo.	Rota entre las cuatro estrategias, para poder compararlas en vivo sin reiniciar.

include/Renderer.h

Funcion / tipo	Entradas	Salidas	Descripcion
struct CameraState	yawDegrees, pitchDegrees : float.	No aplica (tipo de dato).	Orientacion de la camara en orbita alrededor de la piramide. El giro es puramente visual: no toca el estado de la simulacion, de modo que los tiempos de fisica y las mediciones de speedup no se ven afectados por la animacion de la camara.

src/Renderer.cpp

Funcion / tipo	Entradas	Salidas	Descripcion
CameraBasis cameraBasisFor(float yawDegrees, float pitchDegrees)	yawDegrees, pitchDegrees : float — angulos de la camara.	CameraBasis — ejes horizontal y vertical de la pantalla en coordenadas de mundo.	Invierte la rotacion de la vista, que es $R_x(\text{pitch}) * R_y(\text{yaw})$, para obtener los ejes sobre los que se construye cada billboard. Sin esto, los cuadrilateros dejarian de mirar a la camara en cuanto esta girara.
GLuint createPegTexture()	No aplica.	GLuint — identificador de la textura creada.	Genera la textura exclusiva de las clavijas: la misma esfera iluminada que las pelotas, pero con un reborde oscuro marcado y un realce especular mucho mas tenue. Junto con el mayor tamano, los tonos frios y la ausencia de halo, es lo que impide que una clavija se confunda con una pelota.
void renderPyramidRings(const std::vector& pegs, float time)	pegs : const std::vector& — clavijas ordenadas por nivel; time : float — reloj.	Dibuja un aro por cada nivel de la piramide.	Une con una linea las clavijas de cada nivel. Sin estos aros la estructura se lee como una nube de esferas sueltas; con ellos la piramide escalonada se reconoce de inmediato, incluso cuando las pelotas cubren parte de ella.
void renderGround(const SimulationParams& params)	params : const SimulationParams& — dimensiones de la escena.	Dibuja el disco del piso y sus anillos concentricos.	El piso da la referencia de profundidad que hace evidente el giro de la camara.
void renderBins(const SimulationParams& params, const std::vector& counts)	params : const SimulationParams&; counts : const std::vector& — conteo por sector.	Dibuja la corona de sectores en la base.	Cada sector se levanta en proporcion a las pelotas que recogio y su color pasa de azul a magenta. Es la salida de resultados de la simulacion.

src/main.cpp

Funcion / tipo	Entradas	Salidas	Descripcion
bool saveScreenshot(const std::string& path, int width, int height)	path : const std::string& — ruta destino; width, height : int — tamano del lienzo.	Escribe un archivo BMP. Retorno bool: true si se guardo correctamente.	Lee el framebuffer con glReadPixels e invierte las filas, porque OpenGL las entrega de abajo hacia arriba. Se usa para documentar el proyecto de forma reproducible.

Anexo 3. Bitacora de pruebas

Se realizaron 72 configuraciones distintas, combinando seis valores de N, cuatro modos de ejecucion y hasta cinco cantidades de hilos. Cada configuracion se midio **12 veces**, dos por encima del minimo de diez que exige el enunciado, lo que da un total de 828 mediciones individuales. Todas ellas estan en el archivo docs/resultados/benchmark_muestras.csv; las cifras agregadas estan en docs/resultados/benchmark.csv y la salida completa del banco en docs/resultados/bitacora_benchmark.txt.

El comando que reproduce exactamente la corrida reportada es el siguiente:

```
./build/plinko3d --benchmark --bench-balls 250,500,1000,2000,4000,8000 \  
  --bench-threads 1,2,4,6,8 --bench-reps 12 --bench-steps 240 \  
  --bench-out docs/resultados/benchmark.csv
```

A3.1 Resultados por version

Tabla 6. Version OpenMP con reparto estatico. Cada celda muestra el tiempo promedio por paso, el speedup contra la version secuencial del mismo N y la eficiencia E.

N	1 hilo	2 hilos	4 hilos	6 hilos	8 hilos
250	0.411 ms 1.00x · E=1.00	0.257 ms 1.60x · E=0.80	0.182 ms 2.25x · E=0.56	0.230 ms 1.78x · E=0.30	0.263 ms 1.56x · E=0.20
500	1.099 ms 0.99x · E=0.99	0.615 ms 1.76x · E=0.88	0.358 ms 3.02x · E=0.76	0.369 ms 2.94x · E=0.49	0.425 ms 2.54x · E=0.32
1 000	3.080 ms 1.01x · E=1.01	1.600 ms 1.95x · E=0.97	0.884 ms 3.52x · E=0.88	0.741 ms 4.21x · E=0.70	0.899 ms 3.47x · E=0.43
2 000	10.141 ms 1.00x · E=1.00	5.201 ms 1.94x · E=0.97	2.684 ms 3.76x · E=0.94	2.570 ms 3.93x · E=0.65	2.682 ms 3.76x · E=0.47
4 000	36.237 ms 1.01x · E=1.01	19.108 ms 1.91x · E=0.95	9.467 ms 3.85x · E=0.96	10.308 ms 3.54x · E=0.59	8.766 ms 4.16x · E=0.52
8 000	133.973 ms 1.00x · E=1.00	68.317 ms 1.95x · E=0.98	34.212 ms 3.90x · E=0.98	29.054 ms 4.59x · E=0.77	29.793 ms 4.48x · E=0.56

Tabla 7. Version OpenMP ajustada (region unica, reparto guiado y contadores privatizados).

N	1 hilo	2 hilos	4 hilos	6 hilos	8 hilos
250	0.419 ms 0.98x · E=0.98	0.257 ms 1.60x · E=0.80	0.215 ms 1.91x · E=0.48	0.250 ms 1.64x · E=0.27	0.313 ms 1.31x · E=0.16
500	1.085 ms 1.00x · E=1.00	0.611 ms 1.77x · E=0.89	0.404 ms 2.68x · E=0.67	0.398 ms 2.72x · E=0.45	0.461 ms 2.35x · E=0.29
1 000	3.065 ms 1.02x · E=1.02	1.606 ms 1.94x · E=0.97	0.897 ms 3.48x · E=0.87	0.722 ms 4.31x · E=0.72	0.783 ms 3.98x · E=0.50
2 000	10.184 ms 0.99x · E=0.99	5.193 ms 1.94x · E=0.97	2.739 ms 3.69x · E=0.92	2.019 ms 5.00x · E=0.83	1.987 ms 5.08x · E=0.64
4 000	36.711 ms 0.99x · E=0.99	20.206 ms 1.80x · E=0.90	9.524 ms 3.83x · E=0.96	7.100 ms 5.13x · E=0.86	7.084 ms 5.15x · E=0.64
8 000	134.954 ms 0.99x · E=0.99	69.630 ms 1.92x · E=0.96	38.249 ms 3.49x · E=0.87	31.703 ms 4.21x · E=0.70	26.322 ms 5.07x · E=0.63

Tabla 8. Version secuencial de referencia y version con un std::thread por pelota, con el mejor tiempo de las 12 repeticiones. La eficiencia de esta ultima se calcula sobre N hilos, que es la cantidad que realmente crea.

N	Pasos por repeticion	Secuencial promedio (ms)	Secuencial desv. est.	std::thread promedio (ms)	std::thread speedup	std::thread eficiencia
250	240	0.4107	0.0083	1.5925	0.258x	0.00103
500	240	1.0823	0.0261	4.3213	0.250x	0.00050
1 000	240	3.1168	0.0904	13.2278	0.236x	0.00024
2 000	75	10.0953	1.1005	no viable	—	—
4 000	18	36.4504	0.4459	no viable	—	—
8 000	4	133.4278	3.3241	no viable	—	—

Las celdas marcadas como “no viable” corresponden a valores de N por encima de 1 024, donde el sistema operativo rechaza la creacion de un hilo por pelota. El programa detecta la condicion, la informa y continua con otro modo.

A3.2 Mediciones individuales

Se reproducen a continuacion las doce repeticiones de cada configuracion para dos valores de N representativos: uno pequeno, donde la sobrecarga domina, y uno grande, donde la paralelizacion rinde. Los tiempos estan en milisegundos por paso de fisica.

Tabla 9. Las 12 repeticiones de cada configuracion con N = 500. R1 a R12 son las repeticiones individuales en milisegundos por paso; la columna Min. es el estimador que usa el informe.

Configuracion	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11	R12	Min.	Desv.
Secuencial	1.170	1.090	1.092	1.082	1.084	1.084	1.113	1.140	1.124	1.116	1.114	1.086	1.082	0.026
std::thread x500	4.444	4.564	4.321	4.766	4.787	4.956	4.724	4.398	4.492	4.500	4.528	4.395	4.321	0.185
OpenMP estatico x1	1.132	1.136	1.207	1.164	1.107	1.116	1.121	1.196	1.121	1.099	1.101	1.168	1.099	0.035
OpenMP estatico x2	0.617	0.649	0.797	0.645	0.834	0.631	0.637	0.700	0.615	0.626	0.629	0.631	0.615	0.070
OpenMP estatico x4	0.358	0.378	0.374	0.417	0.412	0.388	0.458	0.510	0.378	0.376	0.380	0.370	0.358	0.042
OpenMP estatico x6	0.390	0.374	0.400	0.387	0.428	0.509	0.393	0.411	0.408	0.383	0.370	0.369	0.369	0.037
OpenMP estatico x8	0.444	0.425	0.434	0.460	0.528	0.479	0.444	0.437	0.562	0.503	0.679	0.496	0.425	0.069
OpenMP ajustado x1	1.178	1.102	1.090	1.099	1.114	1.095	1.157	1.181	1.087	1.085	1.086	1.087	1.085	0.035
OpenMP ajustado x2	0.611	0.619	0.635	0.621	0.615	0.623	0.804	0.864	0.689	0.740	0.782	0.626	0.611	0.085
OpenMP ajustado x4	1.023	0.565	0.450	0.417	0.427	0.405	0.528	0.411	0.404	0.429	0.418	0.418	0.404	0.167
OpenMP ajustado x6	0.426	0.439	0.434	0.411	0.600	0.461	0.459	0.456	0.463	0.436	0.398	0.418	0.398	0.049
OpenMP ajustado x8	0.476	0.517	0.478	0.473	0.464	0.498	0.465	0.461	0.475	0.492	0.495	0.474	0.461	0.016

Tabla 10. Las 12 repeticiones de cada configuracion con N = 8 000. R1 a R12 son las repeticiones individuales en milisegundos por paso; la columna Min. es el estimador que usa el informe.

Configuracion	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11	R12	Min.	Desv.
Secuencial	136.5	141.0	136.0	145.4	133.4	137.4	134.0	135.6	140.1	134.8	135.6	134.8	133.4	3.324
OpenMP estatico x1	143.3	136.3	141.2	138.3	137.4	134.0	137.5	136.5	135.5	137.1	143.6	135.1	134.0	2.997
OpenMP estatico x2	70.28	68.51	74.13	75.80	75.86	83.91	74.48	70.58	75.97	68.32	73.37	70.28	68.32	4.159
OpenMP estatico x4	34.78	34.57	34.65	36.77	40.36	36.46	41.91	35.34	34.21	34.49	34.31	35.47	34.21	2.397
OpenMP estatico x6	29.05	30.97	32.89	32.43	30.01	31.69	31.91	32.51	32.07	31.54	30.68	34.14	29.05	1.296
OpenMP estatico x8	31.92	34.83	38.09	31.83	30.03	29.99	32.48	31.31	32.79	29.79	34.93	38.49	29.79	2.843
OpenMP ajustado x1	135.0	136.7	141.8	142.4	138.7	146.1	229.8	142.2	138.4	139.0	136.9	137.8	135.0	25.12
OpenMP ajustado x2	70.36	69.63	71.38	73.45	94.27	77.12	80.19	123.5	102.1	82.15	73.33	72.52	69.63	15.64
OpenMP ajustado x4	85.12	88.65	67.33	55.76	47.94	64.68	49.16	43.27	44.14	38.25	47.50	63.44	38.25	15.63
OpenMP ajustado x6	53.68	41.57	82.88	45.05	31.70	32.33	35.56	56.24	31.87	42.79	37.97	35.37	31.70	14.08
OpenMP ajustado x8	223.4	31.09	28.06	31.26	28.81	26.77	46.83	37.33	45.71	26.32	33.08	31.46	26.32	52.93

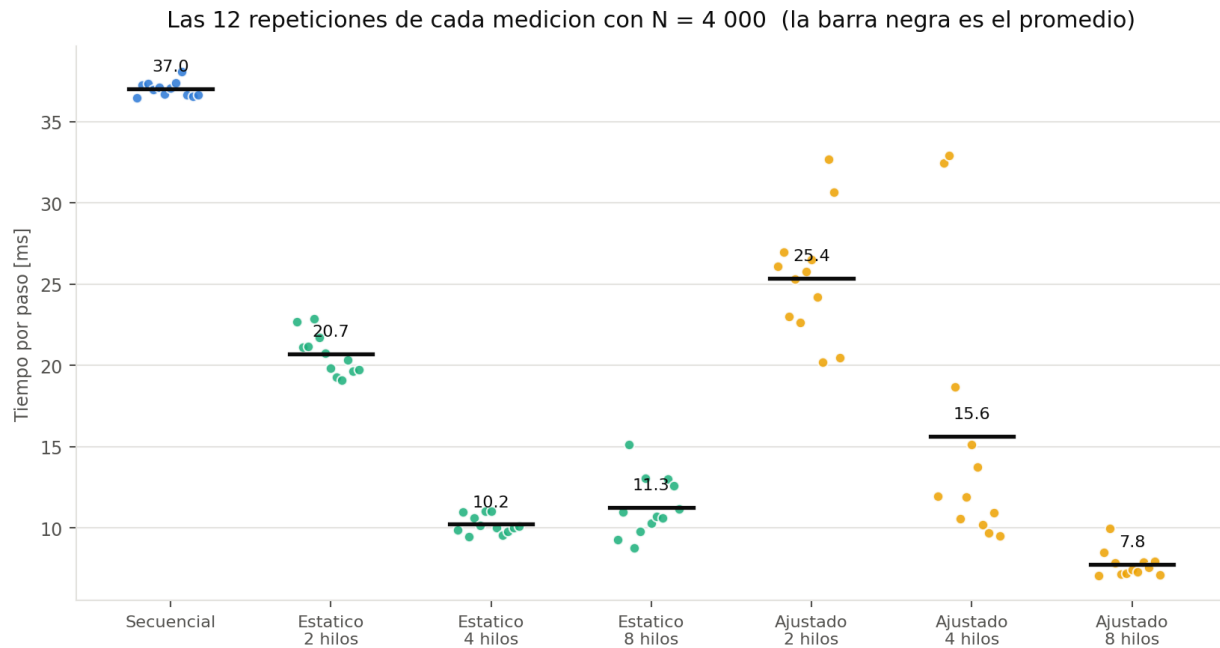


Figura 10. Captura grafica de las mediciones con N = 4 000: cada punto es una de las doce repeticiones y la barra negra marca el promedio.

A3.3 Captura de la salida del banco de pruebas

Se reproduce un fragmento textual de la salida por consola del banco de pruebas, correspondiente a la carga mayor evaluada.

```
== N = 8000 (4 pasos por repeticion) ==
secuencial      137.0341 ms/paso (min 133.4278 max 145.3695 sd 3.3241)      7.3 FPS
std::thread x8000 no aplicable: N supera el limite practico de un hilo por pelota
openmp-static x1 137.9751 ms/paso (min 133.9733 max 143.6062 sd 2.9973) speedup 0.
99x eficiencia 0.993
openmp-static x2 73.4581 ms/paso (min 68.3167 max 83.9059 sd 4.1590) speedup 1.
87x eficiencia 0.933
openmp-static x4 36.1090 ms/paso (min 34.2122 max 41.9068 sd 2.3975) speedup 3.
80x eficiencia 0.949
openmp-static x6 31.6590 ms/paso (min 29.0540 max 34.1362 sd 1.2959) speedup 4.
33x eficiencia 0.721
openmp-static x8 33.0394 ms/paso (min 29.7933 max 38.4908 sd 2.8426) speedup 4.
15x eficiencia 0.518
openmp-tuned x1 147.0683 ms/paso (min 134.9542 max 229.8084 sd 25.1224) speedup 0.
93x eficiencia 0.932
openmp-tuned x2 82.4948 ms/paso (min 69.6298 max 123.4516 sd 15.6400) speedup 1.
66x eficiencia 0.831
openmp-tuned x4 57.9360 ms/paso (min 38.2490 max 88.6469 sd 15.6255) speedup 2.
37x eficiencia 0.591
openmp-tuned x6 43.9193 ms/paso (min 31.7033 max 82.8833 sd 14.0838) speedup 3.
12x eficiencia 0.520
openmp-tuned x8 49.1785 ms/paso (min 26.3218 max 223.4058 sd 52.9302) speedup 2.
79x eficiencia 0.348
```

Fragmento de docs/resultados/bitacora_benchmark.txt.

A3.4 Resultado de las pruebas automatizadas

Ademas de las mediciones de rendimiento, la entrega incluye cuatro suites de pruebas de correccion que se ejecutan con `ctest --test-dir build --output-on-failure`.

```
Test project /Users/javiervalladares/Library/Mobile Documents/com~apple~CloudDocs/Octavo Semestre/
Paralela/Proyecto1/build
  Start 1: equivalencia_modos
1/4 Test #1: equivalencia_modos ..... Passed    0.43 sec
  Start 2: determinismo_hilos
2/4 Test #2: determinismo_hilos ..... Passed    0.33 sec
  Start 3: validacion_argumentos
3/4 Test #3: validacion_argumentos ..... Passed    0.00 sec
  Start 4: conservacion_conteos
4/4 Test #4: conservacion_conteos ..... Passed    0.30 sec
100% tests passed, 0 tests failed out of 4
Total Test time (real) =  1.07 sec
```

Salida de CTest en la version entregada.

Anexo 4. El programa en ejecucion

Las capturas siguientes se tomaron con la opcion `--screenshot` del propio programa, que vuelca el framebuffer despues de un numero fijo de cuadros. Eso las hace reproducibles: la misma semilla y el mismo numero de cuadros producen la misma imagen.

El HUD de la esquina superior izquierda es el despliegue de resultados en vivo que pide el enunciado. Muestra los cuadros por segundo —en verde por encima de 58, en ambar entre 30 y 58 y en rojo por debajo—, el modo de ejecucion activo, N, la cantidad de hilos, el tiempo de fisica y el tiempo total por cuadro, el speedup estimado contra el modo secuencial y el total de pelotas recicladas. Las barras magenta que rodean la base son el histograma de los sectores contadores.

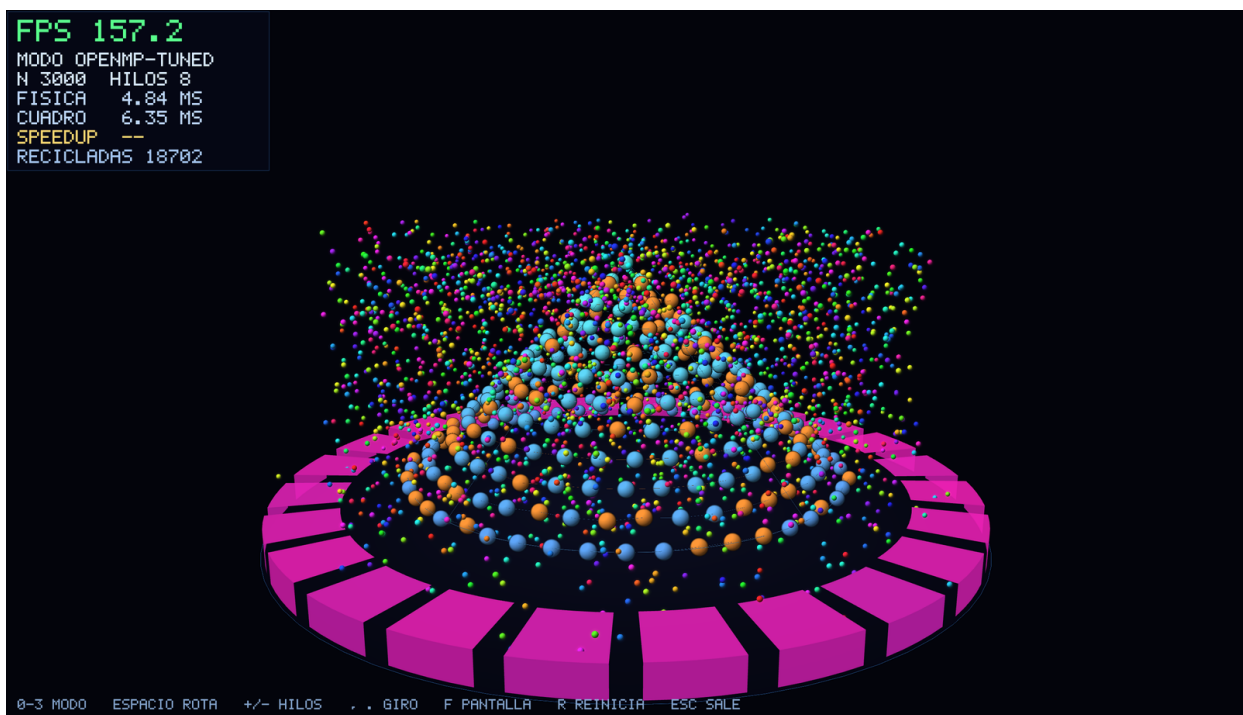


Figura 11. Configuración por omisión: $N = 3\,000$ pelotas cayendo sobre una pirámide de 277 clavijas repartidas en 12 niveles, con 24 sectores contadores en la base. Las clavijas grandes en tonos fríos forman la estructura; las amarillas son las que oscilan.

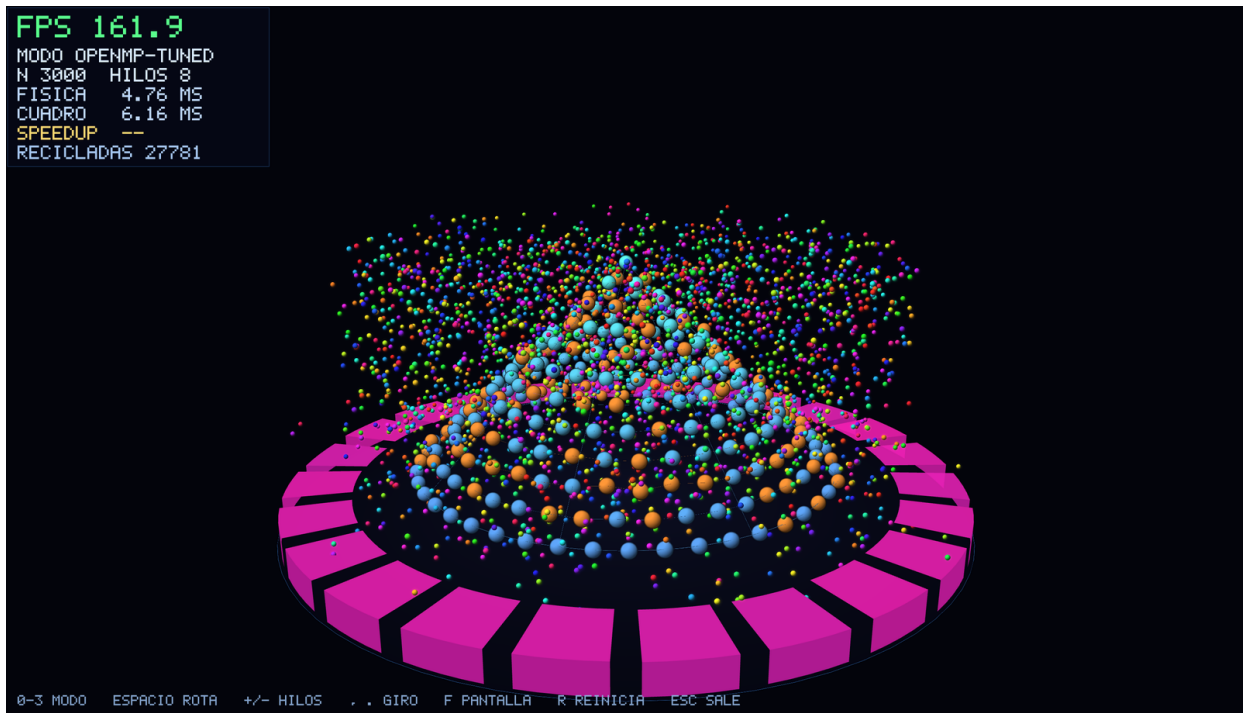


Figura 12. La misma escena unos segundos despues. La camara ha girado alrededor del eje vertical, lo que revela la forma conica y la corona de sectores. El giro es puramente visual y no interviene en la fisica.

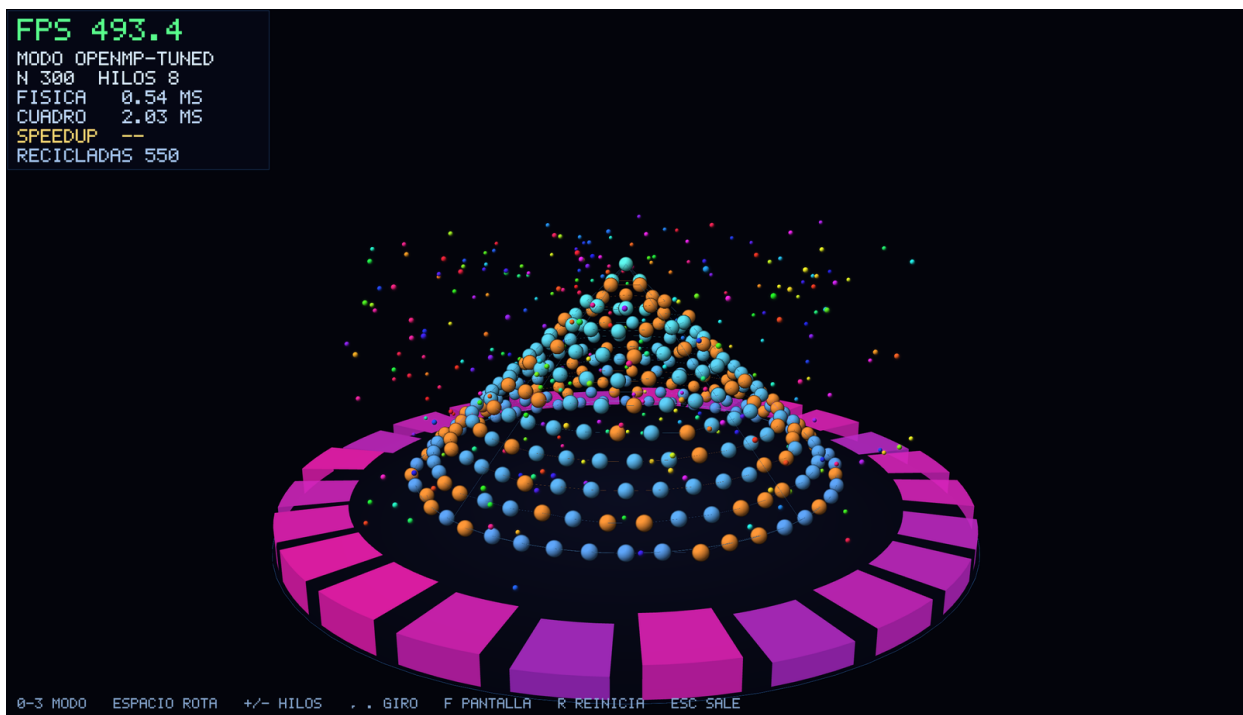


Figura 13. La misma piramide con solo 300 pelotas, para que se aprecie la estructura: los aros que unen las clavijas de cada nivel y las aristas que van del vertice a la base.